

BIBLIA - Hindi Edition

Page 1

आधुनिक कृत्रिम बुद्धिमत्ता

BIBLIA

AI का आधुनिक उपयोग, subagents, SDD, AGENTS.md,
GitHub, prompt engineering और harness engineering.
लेखकत्व

Nicolás Ezequiel Melluso

nicolas.e.melluso@gmail.com

linkedin.com/in/nicolas-ezequiel-melluso

github.com/Nicolas-Melluso

BIBLIA - Nicolás Ezequiel Melluso

1/92

सामान्य अनुक्रमणिका

01

कृत्रिम बुद्धिमत्ता का आधुनिक उपयोग

मॉडल से चैट करने से लेकर सोच, नष्टिपादन और सत्यापन की प्रणाली के साथ काम करने तक

02

सबएजेंट्स का बुद्धिमान उपयोग

बेहतर डेलीवेशन के लिए मानदंड, पैटर्न और ऑपरेशनल क्लोज़र

03

SDD और सपोर्ट संरचना

AI-सक्षम repositories में लागू Specification-Driven Development

04

AGENTS.md, .github और Prompt-स्टाइल Commands

agents, SDD, Copilot, workflows और reusable prompts के लिए तैयार repository कैसे बनाएं

05

Prompt Engineering और Harness Engineering

अलग-अलग prompts से versioned, evaluable और production-ready systems तक

BIBLIA - Nicolás Ezequiel Melluso

2/92

खंड 01

कृत्रिम बुद्धिमत्ता का आधुनिक उपयोग

मॉडल से चैट करने से लेकर सोच, नष्टिपादन और सत्यापन की प्रणाली के साथ काम करने तक

BIBLIA - Nicolás Ezequiel Melluso

3/92

केंद्रीय विचार

AI का आधुनिक उपयोग यह नहीं है कि 'चैत खोलो, "यह कर दो" कहो और पहली प्रतिक्रिया मान लो। वह पहला चरण था। आधुनिक चरण यह है कि AI को काम की एक परत की तरह माना जाए: सहायक, तकनीकी साथी, शोधकर्ता, नष्टपादक, समीक्षक और मेमोरी सिस्टम का संयोजन। फर्क लंबे prompt लिखने में नहीं है। फर्क उस फ्लो को डिजाइन करने में है जहाँ हर इंटरैक्शन पुनः उपयोग योग्य context, artifacts, tests और decisions छोड़कर जाए।

AI का अपरपिक्व उपयोग बातचीत-आधारित और disposable होता है: कुछ पूछा, जवाब मिला, थोड़ा कॉपी किया और आगे बढ़ गए। परपिक्व उपयोग operational होता है: उद्देश्य तय करो, context दो, constraints स्पष्ट करो, काम को हिससों में बाँटो, परिणाम सत्यापित करो और जो सीखा उसे दर्ज करो। मूल्य तब आता है जब AI सिर्फ टेक्स्ट मशीन न रहकर development, research या production प्रक्रिया का विस्तार बन जाता है।

यह खंड एक सरल कार्य-मॉडल प्रस्तावित करता है: AI को चार परतों वाली प्रणाली की तरह सोचो।

1. Context layer: कार्रवाई से पहले AI को क्या जानना चाहिए।

2. Task layer: अभी उसे क्या उत्पन्न करना है।

3. Verification layer: कैसे पता चले कि परिणाम काम का है।

4. Memory layer: कहाँ दर्ज रहे ताकि विही reasoning फिर न दोहरानी पड़े।

जब ये चारों परतें मौजूद हों, AI गंभीर कामों में मदद कर सकता है: specifications लिखना, code review करना, विकल्पों की तुलना करना, documentation बनाना, tests चलाना, risks पहचानना, presentations तैयार करना, किसी व्यक्ति को train करना या technical decisions तेज करना। जब ये न हों, AI कभी-कभी बहुत प्रभावशाली और चुपचाप खतरनाक हो जाता है।

व्यवहार में क्या बदला

महत्वपूर्ण बदलाव सिर्फ मॉडल के बेहतर होने का नहीं है। बदलाव यह है कि अब हम ऐसे agents के साथ काम कर सकते हैं जो tools से जुड़े हों: editor, terminal, browser, repository, issue tracker, documentation, local databases, tests, linters, emulators और pipelines। पहले मॉडल काम के बाहर से जवाब देता था। अब वह काम के भीतर भाग ले सकता है।

इससे मांग रखने का तरीका बदलना पड़ता है। आधुनिक अनुरोध सिर्फ "X समझाओ" नहीं कहता। वह कहता है: "इन files को पढ़ो, current behavior पहचानो, सीमांति बदलाव प्रस्तावित करो, उसे implement करो, ये tests चलाओ और risks सहित summary दो।"

AI अब oracle नहीं, contract के तहत operator बन जाता है।

मानव भूमिका भी बदलती है। अब जीत इस बात से नहीं मिलती कि किसने सब कुछ हाथ से टाईप किया। जीत अच्छे context, अच्छे constraints, अच्छे acceptance criteria और अच्छे verification mechanisms तय करने से मिलती है। AI बहुत कुछ बना सकता है, पर वह अपने आप नहीं जानता कि आपके business के लिए कौन-सा tradeoff सही है, कौन-सा legal risk स्वीकार्य है, कौन-सा technical debt सहना है, या आप कौन-सा user experience बचाना चाहते हैं।

आधुनिक प्रश्न यह नहीं है "कौन-सा prompt उपयोग करूँ।" आधुनिक प्रश्न है "ऐसी कौन-सी कार्य-प्रणाली है जो हर हफ्ते परिणाम बेहतर बनाती है।"

अनुशासित कार्य-चक्र

AI के साथ एक मजबूत फ्लो ऐसा देख सकता है:

BIBLIA - Nicolás Ezequiel Melluso

Intent -> Context -> Plan -> Execution -> Verification -> Record -> Next iteration

Intent इच्छति परिणाम तय करता है। Context अस्पष्टता घटाता है। Plan आवेगपूर्ण काम से बचाता है। Execution ठोस artifacts देता है। Verification वश्वसनीय देखने वाली और वास्तव में सही चीज़ में अंतर करता है। Record चैट में ज्ञान खोने से बचाता है। अगला iteration काम को संचति सीख में बदल देता है।

एक सरल उदाहरण:

Intent:

मैं magic link authentication जोड़ना चाहता हूँ।

Context:

Node/TypeScript repo, PostgreSQL, service architecture, Vitest tests।

Plan:

1. मौजूदा auth ढूँढो।
2. token table जोड़ो।
3. service implement करो।
4. unit और integration tests जोड़ो।
5. environment variables document करो।

Verification:

npm test

npm run typecheck

login -> link -> session फ्लो का manual test

Record:

छोटा ADR: password के बजाय magic link क्यों।

expected behavior की spec।

rollback checklist।

यह फ्लो किसी एक खास tool पर निर्भर नहीं है। यह Codex, Copilot, Cursor, Claude Code, Gemini CLI या custom agent के साथ काम करता है। परपिक्वता प्रक्रिया में है।

Context की न्यूनतम इकाई

AI तब बेहतर काम करता है जब उसे पैकेज्ड context मलि, बखिरी हुई जानकारी नहीं। गंभीर कार्य के लिए context की न्यूनतम इकाई में यह शामिल होना चाहिए:

BIBLIA - Nicolás Ezequiel Melluso

5/92

तत्त्व

कसिलिए

उदाहरण

उद्देश्य

गलत चीज optimize होने से रोकता है

"abandoned checkout errors कम करना"

वर्तमान स्थिति

शुरुआती बढ़ि देता है

"Stripe return /checkout/success पर आता है"

constraints

समाधानों को सीमिति करता है

"prices न छेड़ें, provider migrate न करें"

relevant files

blind exploration कम करता है

src/server.js , public/app.js

acceptance criteria

closure परभाषित करता है

"payment से लौटे तो purchased state देखि"

verification

testing अनिवार्य करता है

"local smoke test और unit test"

risks

नाजुक हसिसों को visible करता है

"logs में secrets leak न हों"

इस न्यूनतम इकाई के बिना मॉडल assumptions से खाली जगह भरता है। कभी सही होता है। वास्तविक systems में कभी-कभी वह ऐसी logic पर चलता है जो उचित लगती है पर प्रोजेक्ट की नहीं होती, और चीजें तोड़ देता है।

AI के लिए अच्छे कार्य

AI खास तौर पर तब मजबूत है जब वह explicit जानकारी पर काम कर सके और output सत्यापित हो सके। कुछ high-value उपयोग:

1. मौजूदा code का सार और map बनाना।
2. किसी idea को reviewable specification में बदलना।
3. acceptance criteria से test cases बनाना।
4. docs, code और tests में inconsistencies ढूँढना।
5. test suite के साथ सीमिति क्षेत्र का refactor करना।
6. migration या validation scripts तैयार करना।
7. repeatable operations के लिए runbooks बनाना।
8. concrete rules के साथ PRs review करना।
9. conversations को actionable tasks में बदलना।
10. किसी और व्यक्ति के लिए training material बनाना।

जब AI से बिना data के निरणय, business policies invent करना, एक साथ system के कई हसिसों को छूना, permissions के बिना infrastructure बदलना, या human review के बिना "final" content बनवाना हो, तब इसकी विश्वसनीयता घटती है। इसका मतलब यह नहीं कि वह मदद नहीं कर सकता। मतलब यह है कि मजबूत control frame चाहिए।

आधुनिक prompt

आधुनिक prompt एक operational brief जैसा होता है। उसे काव्यात्मक या बहुत बड़ा होने की जरूरत नहीं। उसे ambiguity हटानी चाहिए।

BIBLIA - Nicolás Ezequiel Melluso

6/92

Objective:

मैं चाहता हूँ कि तुम इस idea को implementation-ready SDD specification में बदलो।

Context:

प्रोडक्ट claims manage करने वाली एक B2B app है। repo Node.js, TypeScript, PostgreSQL और GitHub Actions उपयोग करता है। हम scope छोटा रखना चाहते हैं।

Input:

Idea: "एक operator को claim दूसरे team को reassign करने देना।"

Constraints:

- अभी पूरी screen design मत करो।
- जरूरी न हो तो नए roles assume मत करो।
- business rules को UI से अलग रखो।
- risks और open questions शामिल करो।

Output:

1. समस्या का सारांश।
2. Functional requirements।
3. Non-functional requirements।
4. Acceptance criteria।
5. Edge cases।
6. 3 slices में implementation plan।
7. Recommended tests।

Quality:

अगर जानकारी कम है तो उसे open question के रूप में चिह्नित करो। business data invent मत करो।

यह format तीन काम करता है: दिशा देता है, सीमाएँ तय करता है और response का evaluation कैसे होगा यह परभाषित करता है।

इसका उद्देश्य मॉडल को "inspire" करना नहीं, बल्कि उसे task के लिए contract करना है।

चैट से repository तक

सबसे महत्वपूर्ण छलांग है ज्ञान को चैट से repository में ले जाना। चैट नाजुक होती है: खो जाती है, अपने आप से टकराती है, versioning खराब होती है, PR में review नहीं होती, और CI में नहीं चलती। repository इसके उलट instructions, specs, decisions, prompts, tests और workflows सुरक्षित रख सकती है।

आधुनिक संगठन आमतौर पर यह वभिजन रखते हैं:

BIBLIA - Nicolás Ezequiel Melluso

7/92

Artifact

Audience

Function

README.md

नए humans

प्रोजेक्ट और शुरुआत का तरीका बताना

AGENTS.md

code agents

operational rules, commands, style और verification

.github/copilot-instructions.md

Copilot

repo responses के लिए general instructions

.github/instructions/*.instructions.md

path-wise Copilot

code के हिस्सों के लिए specific rules

.github/prompts/*.prompt.md

humans और assistants

reusable prompt-style commands

.github/orquestador/sdd/*

टीम और agents

specs, decisions, tasks, traceability

.github/workflows/*

CI/CD

executable automation

व्यावहारिक नियम: जो दोहराया जाता है, उसे file बनना चाहिए। अगर हर बार मदद मांगने पर वही commands, वही style, वही

constraints और वही test criteria फिर समझने पड़ें, तो वह prompt engineering नहीं, context debt है।

छोटी टीम में AI की भूमिकाएँ

भले ही एक tool "assistant" जैसा दखि, roles में सोचना उपयोगी है:

Role

क्या करता है

अच्छा output

Researcher

पढ़ता, तुलना करता, सारांश बनाता, patterns खोजता

file map, risks, questions

Planner

काम को विभाजित करता

slices वाला plan, dependencies, verification

Implementer

files बदलता

scoped patch, tests, summary

Reviewer

errors ढूँढता

file और line के साथ findings

Documenter

काम को knowledge में बदलता

README, ADR, runbook

Evaluator

behavior test करता

commands और results की report

subagents इस separation को औपचारिक बनाते हैं, पर मानसिक मॉडल एक चैट में भी काम करता है। जब एक ही बातचीत सब कुछ

एक साथ करने लगती है, वह भ्रमति हो जाती है। जब हर role की concrete output हो, काम controllable बनता है।

Verification वैकल्पिक नहीं है

AI convincing text और सही दखिने वाला code बना सकता है। गंभीर रक्षा सिर्फ verification है। verification automated हो

सकती है या human, लेकिन होनी चाहिए।

verification के उदाहरण:

1. Unit और integration tests।

BIBLIA - Nicolás Ezequiel Melluso

8/92

2. Typecheck, lint और build।

3. Documented manual smoke test।

4. Acceptance criteria के वरिद्ध तुलना।

5. File-by-file diff review।

6. Real data या fixtures के साथ test।

7. prompts के लिए golden cases से evaluation।

8. Security और permissions checklist।

"अच्छा लग रहा है" पर्याप्त नहीं है। आधुनिक फ्लो मांगता है कि AI बताए: उसने क्या चलाया, क्या नहीं चला सका, क्या बदला, क्या बाकी है और क्या risk शेष है।

उपयोगी मेमोरी, अनंत मेमोरी नहीं

मेमोरी तब उपयोगी है जब वह दोहराव घटाए और consistency बढ़ाए। जब वह लंबे नोट्स का कूड़ाघर बन जाए, तब उपयोगी नहीं रहती। उपयोगी मेमोरी की तीन विशेषताएँ हैं:

1. Retrievable: ज्ञात path में हो।

2. Actionable: decisions, commands, conventions या सीखी गई गलतियाँ शामिल हों।

3. Verifiable: जब repo की current state बदल सकती हो, तब यह उसे replace न करे।

अच्छी मेमोरी के उदाहरण:

- repo context folder के रूप में `.github/orquestador` उपयोग करता है।

- workflows ही executable layer हैं; catalog सिर्फ documentation है।

- runtime changes बंद करने से पहले ``npm test`` और ``npm run build`` चलाओ।

- Windows पर folders rename करने से पहले locks verify करो।

खराब मेमोरी के उदाहरण:

- प्रोजेक्ट महत्वपूर्ण है।

- कभी-कभी fail होता है।

- good practices उपयोग करो।

आधुनिक मेमोरी भावनाएँ नहीं, operations सहेजती है।

30-दिन की adoption योजना

सप्ताह 1: context व्यवस्थिति करें

AGENTS.md बनाओ, वास्तविक commands लिखो, constraints सूचीबद्ध करो और तय करो SDD कहाँ रहेगा। उद्देश्य सब कुछ कवर करना नहीं है। उद्देश्य यह है कि कोई agent repo में आए और आधा घंटा अंदाज़ा लगाने में न गंवाए।

Deliverables:

BIBLIA - Nicolás Ezequiel Melluso

9/92

1. प्रारंभिक AGENTS.md I
2. अपडेटेड README.md I
3. .github/orquestador/context/product.md I
4. .github/orquestador/context/architecture.md I
5. verification commands की सूची।

सप्ताह 2: specifications के आधार पर काम

एक छोटी feature चुनो और implement करने से पहले spec लिखो। acceptance criteria, edge cases, non-goals और tests शामिल करो। फिर AI से सिर्फ एक slice implement करवाओ।

Deliverables:

1. पहली SDD spec I
2. slices वाला plan I
3. relevant technical decision हो तो ADR I
4. संबंधित tests I

सप्ताह 3: reusable prompts लाएँ

दोहराए जाने वाले कार्यों के लिए prompts बनाओ: feature planning, PR review, test generation, ADR writing, runbook preparation I इन्हें .github/prompts या चुने गए orchestration folder में रखो।

Deliverables:

1. plan-feature.prompt.md I
2. review-pr.prompt.md I
3. write-adr.prompt.md I
4. generate-tests.prompt.md I

सप्ताह 4: गुणवत्ता मापें

सरल harnesses या evaluations जोड़ो। बड़ी platform खड़ी करने की जरूरत नहीं। fixtures, expected cases और output compare करने वाले script से शुरुआत करो।

Deliverables:

1. evals/ folder I
2. Input fixtures I
3. Evaluation rubric I
4. Local script या validation workflow I

BIBLIA - Nicolás Ezequiel Melluso

10/92

सामान्य anti-patterns

Anti-pattern

लक्षण

सुधार

हर चीज़ के लिए विशाल prompt

मॉडल कुछ हिससों को ignore करता है

persistent instructions और concrete tasks में बाँटो

verification नहीं

output सुंदर पर fabricated

commands और acceptance criteria परभाषति करो

context सरिफ़ चैट में

हर session में सब दोहरता है

rules repo में लाओ

बहुत व्यापक permissions वाला agent

destructive changes का risk

ownership और minimum permissions

सब कुछ एक subagent में

false parallelism

exploration, implementation और review अलग करो

docs जो चलती नहीं

अच्छी मंशा, पर असर नहीं

docs को workflows और checklists से जोड़ो

AI का आधुनिक उपयोग: checklist

मांग रखने से पहले:

1. अंतमि परिणाम स्पष्ट है।
2. relevant files या domains का नाम बता सकता हूँ।
3. पता है कि क्या नहीं छूना है।
4. verify करने का तरीका है।
5. पूरे system के बजाय पहला slice स्वीकार कर सकता हूँ।

काम के दौरान:

1. risky tasks के लिए छोटे plans मांगता हूँ।
2. research, editing और review अलग रखता हूँ।
3. file ownership बनाए रखता हूँ।
4. बद करने से पहले diffs पढ़ता हूँ।
5. नए decisions दर्ज करता हूँ।

समापन पर:

1. मुझे पता है क्या बदला।
2. कौन-से tests पास हुए पता है।
3. कौन-से tests नहीं चले पता है।
4. कौन-से risks बचे हैं पता है।
5. reusable knowledge files में दर्ज है।

BIBLIA - Nicolás Ezequiel Melluso

11/92

समापन

आधुनिक AI कौशल का स्थापनापन्न नहीं है। जब कौशल व्यवस्थित हो, AI उसे बढ़ाता है। सबसे अधिक मूल्य उसे मलित है जो secret prompt trick नहीं, बल्कि अस्पष्ट काम को verify होने वाली इकाइयों में बदलना जानता है।

अंतिम नियम सरल है: अगर output महत्वपूर्ण है, तो AI को production system का हिस्सा मानो। उसे context, limits, tools, tests और memory दो। बाकी सब सिर्फ चैट है।

BIBLIA - Nicolás Ezequiel Melluso

12/92

खंड 02

सबएजेंट्स का बुद्धिमान उपयोग

बेहतर डेलीवेशन के लिए मानदंड, पैटर्न और ऑपरेशनल क्लोजर

BIBLIA - Nicolás Ezequiel Melluso

13/92

यह वॉल्यूम कसि काम का है

आधुनिक agents और subagents के साथ काम करना कामों को यूँ ही बाँट देना नहीं है। उपयोगी डेलीगेशन और समय की बर्बादी के बीच अंतर आम तौर पर तीन चीजों में होता है: समस्या का आकार, फ्रेमिंग की गुणवत्ता, और परणाम पर नियंत्रण का स्तर। जब यह सही तरीके से किया जाता है, काम तेज़ होता है, context बेहतर उपयोग होता है, और main thread उन परिणयों के लिए खाली रहता है जिनमें सच में मानवीय या आर्कटिकचरल जजमेंट चाहिए।

यह दस्तावेज़ तकनीकी प्रोजेक्ट्स में subagents उपयोग करने का एक व्यावहारिक तरीका प्रस्तावित करता है। इसका उद्देश्य किसी जादू या पूर्ण ऑटोमेशन को बेचना नहीं है। लक्ष्य अधिक विनिर्भर और अधिक उपयोगी है: आप काम को समझदारी से बाँट सकें, हर subagent से सिर्फ़ जरूरी चीज़ माँग सकें, traceability बनाए रखें, और हर चरण को evidence के साथ बंद करें।

मुख्य विचार यह है: subagent सोच का विकल्प नहीं है। यह सीमिति execution unit है जो न्यूनतम context, स्पष्ट ownership और verify की जा सकने वाली output के साथ काम करता है। इनमें से कोई भी हिससा गायब हो, तो अभी डेलीगेट न करना बेहतर है।

कब डेलीगेट करना चाहिए

हर काम subagent के लायक नहीं होता। बहुत जल्दी डेलीगेट करने से शोर, दोहराव और ऐसे जवाब मिलते हैं जो सही लगते हैं पर समस्या हल नहीं करते। बहुत देर से डेलीगेट करने पर आप ऐसे दोहराए जाने वाले काम हाथ से करते रहते हैं जो parallel में हो सकते थे। एक अच्छा व्यावहारिक नियम है: तब डेलीगेट करें जब नीचे की कई शर्तें पूरी हों:

कार्य स्पष्ट रूप से सीमिति हो।

अपेक्षित परिणाम verify किया जा सके।

कार्य को सिस्टम के अन्य हिस्सों के साथ cross decisions की जरूरत न हो।

समन्वय की लागत सही ठहराने लायक पर्याप्त repetitive, exploratory या mechanical काम हो।

आवश्यक context कुछ निर्देशों और ठोस फाइलों में समा जाए।

subagent के sensitive चीज़ छूने का जोखिम ownership या read-only से नियंत्रित हो।

डेलीगेशन के अच्छे उदाहरण:

यह ढूँढना कि कोई behavior कहाँ implement है।

मॉड्यूल्स के बीच data flow मैप करना।

कोड के किसी क्षेत्र के लिए existing tests पहचानना।

नोट्स या पहले से बनी संरचना से दस्तावेज़ का draft बनाना।

सीमिति फोल्डर पर targeted validation चलाना।

ऐसी technical hypotheses जाँचना जिनमें एक साथ बहुत हिस्से edit न करने पड़ें।

डेलीगेशन के खराब उदाहरण:

“पूरा सिस्टम ठीक कर दो”।

“आर्कटिकचर फिर से बना दो”।

“repo में जाओ और जो देखें उसे बेहतर कर दो”।

“acceptance criteria के बनें product decisions ले लो”।

baseline, metric और scope के बनें “performance optimize कर दो”।

BIBLIA - Nicolás Ezequiel Melluso

14/92

अगर आप यह नहीं बता पा रहे कौन-सी file या कौन-सी output चाहिए, तो संभव है कि आप अभी डेलीगेट करने के लिए तैयार नहीं हैं।

Explorer और worker

subagent workflow में सबसे उपयोगी वभिजन explorer और worker का है।

Explorer

explorer समझने के लिए होता है। इसका काम पढ़ना, मैप करना, तुलना करना, और condensed जानकारी लौटाना है। इसे बदलाव नहीं करने चाहिए, जब तक task स्पष्ट रूप से exploration-with-annotation न हो; और तब भी डिफ़ॉल्ट read-only रखना बेहतर है।

सामान्य उपयोग:

implementations ढूँढना।

references ट्रेस करना।

existing patterns का सार निकालना।

relevant tests, scripts या configurations पहचानना।

code छुए बिना विकल्पों की तुलना करना।

इससे क्या माँगना है:

concrete files quote करें;

short bullets में summary दें;

अनावश्यक solutions न propose करें;

uncertainties पहचान करें;

क्या confirm नहीं कर पाया, यह बताएं।

जब आपको अभी change surface का वास्तविक आकार नहीं पता, तब explorer बहुत मूल्यवान होता है। edit करने से पहले map समझें।

Worker

worker काम करने के लिए होता है। इसे सीमिति objective, स्पष्ट editing zone और verify की जा सकने वाली output criteria मिलती है। यह files edit कर सकता है, commands चला सकता है, या patch तैयार कर सकता है, लेकिन हमेशा explicit ownership के भीतर।

सामान्य उपयोग:

एक specific function implement करना।

validation script बनाना।

documentation file समायोजित करना।

working branch या files के subset पर hypothesis टेस्ट करना।

fixtures या test data तैयार करना।

इससे क्या माँगना है:

सिर्फ authorized files छुएं;

BIBLIA - Nicolás Ezequiel Melluso

15/92

दूसरों के changes revert न करे;

क्या बदला, यह समझाए;

concrete commands से validate करे;

काम review-ready छोड़े।

सामान्य नियम सरल है: uncertainty घटाने के लिए explorer, पहले से समझे काम को execute करने के लिए worker।

न्यूनतम context

डेलीगेशन में सबसे आम गलतियों में से एक है बहुत ज्यादा context देना। subagent को “सब कुछ” देना सुरक्षित लगता है, लेकिन अक्सर गुणवत्ता घटती है। context जतिना बढ़ेगा, शोर उतना बढ़ेगा, लागत बढ़ेगी, और irrelevant signals मिलाने की संभावना बढ़ेगी।

अच्छा minimal context इन सवालों का जवाब देता है:

1. आप क्या हासिल करना चाहते हैं।
2. सिस्टम का कौन-सा हिस्सा वह छू सकता है।
3. कौन-सी files या paths relevant हैं।
4. उसे क्या नहीं छूना चाहिए।
5. कैसे पता चलेगा कि वह सही तरह से पूरा हुआ।

उपयोगी framing कुछ ऐसी हो सकती है:

Objective: एक ठोस वाक्य।

Scope: files और folders।

Constraints: X न छूना, Y modify न करना, Z के बाहर behavior न बदलना।

Expected output: summary, patch, findings list या script।

Verification: tests, commands या manual review।

पूरा project history समझाना ज़रूरी नहीं है। वह हिस्सा समझाना ज़रूरी है जिसकी subagent को ज़रूरत है ताकि वह बिना improvisation के काम कर सके।

क्या शामिल करे

सटीक समस्या।

output format।

ownership तय करने वाली files।

validation commands।

acceptance criteria।

क्या टालें

ऑपरेशनल relevance के बिना लंबी कहानियाँ।

विरिधाभासी राय।

पुरानी clues जो अब लागू नहीं होती।

मानसिक snapshots जिन्हें agent verify नहीं कर सकता।

BIBLIA - Nicolás Ezequiel Melluso

16/92

“इसे काफी बेहतर कर दो” जैसे खुले निर्देश।

अगर subagent को गलती से बचने के लिए key clarification चाहिए, तो रुककर reframing करना बेहतर है। अगर उसे सिर्फ noisy details चाहिए, तो वे मत दें।

File ownership

ownership वह चीज़ है जो कई agents को एक ही जगह पर चढ़ने से रोकती है। हर task की साफ सीमा होनी चाहिए: worker कौन-सी files edit कर सकता है, कौन-सी सिर्फ पढ़ सकता है, और सिस्टम के कौन-से हिस्से off-limits हैं।

यह सिर्फ hygiene नहीं है। यह parallel काम करते समय integrity बनाए रखने का तरीका है।

अच्छी assignment में शामिल होता है:

authorized file या folder;

permission type: read, edit या inspection-only;

impact limits;

क्या change invasive माना जाएगा, उसका criterion;

पुष्टिक्रिये दूसरों का काम revert नहीं करेगा।

अच्छे ownership का उदाहरण:

Worker A: src/docs/intro.md और src/docs/glosario.md , सिर्फ content edits।

Worker B: scripts/validate-docs.mjs , सिर्फ यह file और इसका associated test।

Explorer C: src/ के नीचे कोई भी file, पर बर्ना edits।

खराब ownership का उदाहरण:

“जो ज़रूरी लगे edit कर दो”।

“पूरा module ठीक-ठाक कर दो”।

“देखो कुछ बेहतर मलें तो कर दो”।

जब ownership स्पष्ट होता है, review भी बेहतर होता है। आपको पता होता है क्या बदला, क्यों बदला, और क्या scope के बाहर है।

Parallelism

subagents तब चमकते हैं जब आप independent काम बाँट सकते हैं। parallelism का मतलब चर्चा में एक साथ ज्यादा काम करना नहीं, बल्कि ऐसे काम अलग करना है जो एक-दूसरे को block न करें।

तीन उपयोगी सूत्र हैं:

Exploration parallelism

कई explorers अलग-अलग जानकारी parallel में खोजते हैं।

उदाहरण:

एक implementation locate करता है;

दूसरा tests पहचानता है;

BIBLIA - Nicolás Ezequiel Melluso

17/92

तीसरा similar patterns summarize करता है;

चौथा risks या dependencies खोजता है।

यह बड़े task की शुरुआत में बहुत मदद करता है। सब कुछ sequentially पढ़ने के बजाय, जल्दी map मलित है।

Execution parallelism

कई workers अलग क्षेत्रों में बदलाव करते हैं, बशर्ते files overlap न हों।

उदाहरण:

एक documentation edit करता है;

दूसरा validation adjust करता है;

तीसरा examples या fixtures तैयार करता है।

इसमें अनुशासन चाहिए। अगर दो workers files साझा करते हैं, तो parallelism का आधार टूट जाता है और merges या rewrites पर टकराव शुरू हो जाता है।

Verification parallelism

एक subagent implement करता है और दूसरा बाहर से verify करता है।

उदाहरण:

worker A एक function बदलता है;

worker B जाँचता है कि relevant tests मौजूद हैं और change conventions नहीं तोड़ता;

main thread परणाम integrate करता है।

यह pattern production और evidence अलग करने में उपयोगी है। independent verification confirmation bias कम करती है।

कब parallelize न करें

parallelize करना उचित नहीं जब:

एक task का निरणय दूसरे के परणाम पर निर्भर हो;

एक ही file कई agents edit करेंगे;

architecture अभी चर्चा में हो;

coordination लागत, बचत से अधिक हो;

एक गलती एक साथ कई हिससों को contaminate कर सकती हो।

parallelize करना अपने आप में लक्ष्य नहीं है। यह तभी tool है जब independence सच में मौजूद हो।

Anti-patterns

anti-patterns शुरुआत में productivity जैसे लगते हैं और बाद में operational debt बन जाते हैं।

1. Vague delegation

आप बहुत व्यापक मांग रखते हैं और बहुत generic output पाते हैं। agent खाली जगह assumptions से भर देता है।

BIBLIA - Nicolás Ezequiel Melluso

18/92

सामान्य संकेत: जवाब साफ-सुथरा लगता है पर concrete files या decisions पर नहीं उतरता।

2. Context dumping

आप पूरा repo, पूरी chat, सारे notes दे देते हैं। agent focus खो देता है।

सामान्य संकेत: लंबे जवाब जिनमें relevant हिस्से शोर के साथ मिले हों।

3. Diffuse ownership

किसी को नहीं पता कौन क्या छू सकता है। cross edits, overwrite और confusion शुरू हो जाते हैं।

सामान्य संकेत: "मुझे लगा वह folder free था"।

4. Subagents doing design

subagent strategy गढ़ना शुरू कर देता है जबकि उसे सिर्फ एक concrete slice execute करनी थी।

सामान्य संकेत: मांगा गया change पूरा करने से पहले पूरे सिस्टम को restructure करने का प्रस्ताव।

5. False parallelism

कई tasks "parallel" शुरू होते हैं पर वास्तव में एक ही जोन के लिए प्रतिस्पर्धा करते हैं।

सामान्य संकेत: file conflicts, inconsistent results, या काम दोबारा करना पड़ना।

6. Closure without evidence

subagent कहता है कि काम पूरा है, पर validate कैसे होगा यह नहीं दिखाता।

सामान्य संकेत: न command, न file, न acceptance criteria।

7. Mixing reading with editing without control

agent explore भी करता है और "जब कर ही रहा था" कहकर scope से बाहर चीजें भी बदल देता है।

सामान्य संकेत: अनचाहे collateral changes।

golden rule सख्त लेकिन सरल है: अगर किसी task की स्पष्ट review नहीं हो सकती, तो संभवतः delegation गलत हुआ।

Model और effort matrix

हर subagent को एक जैसा model type या reasoning level नहीं चाहिए। व्यावहारिक matrix से सोचना उपयोगी है: task

complexity बनाम required effort।

वचिर नाम याद करना नहीं, बल्कि काम के अनुसार उचित संयोजन चुनना है।

Mechanical tasks

उदाहरण: file inspection, search, pattern extraction, simple validations, formatting।

Suggested model: lightweight या fast।

Effort: low।

Objective: गति और कम लागत।

BIBLIA - Nicolás Ezequiel Melluso

19/92

Synthesis tasks

उदाहरण: findings summarize करना, options compare करना, documentation condense करना, flow map करना।

Suggested model: lightweight या medium, material density के अनुसार।

Effort: low से medium।

Objective: output को अनावश्यक बड़ा किए बिना जानकारी व्यवस्थिति करना।

Bounded implementation tasks

उदाहरण: एक file edit करना, script बनाना, test adjust करना।

Suggested model: medium।

Effort: medium।

Objective: अत्यधिक लागत के बिना अच्छा technical judgment।

Complex debugging tasks

उदाहरण: multiple-cause bugs, cross integration, intermittent failures।

Suggested model: stronger model।

Effort: medium से high।

Objective: अधिक reasoning क्षमता, कम shortcuts।

Final review tasks

उदाहरण: महत्वपूर्ण patch review करना, regressions पकड़ना, assumptions पर सवाल करना।

Suggested model: stronger, या कम से कम implement करने वाले से अलग।

Effort: medium से high।

Objective: independence और critical नज़रिया।

Practical rule

अगर task repetitive है, heavy model पर खर्च न करें।

अगर task fine criteria पर निर्भर है या कई हिससों से जुड़ा है, level बढ़ाएँ।

अगर result महत्वपूर्ण delivery तय करेगा, independent review जोड़ें।

effort "हमेशा high" नहीं होना चाहिए। उसे risk और ambiguity के साथ चलना चाहिए।

Delegation prompts के उदाहरण

बेहतरीन subagent prompts खुले अनुरोध जैसे नहीं लगते। वे अच्छे लिखे tickets जैसे लगते हैं।

BIBLIA - Nicolás Ezequiel Melluso

20/92

Implementation exploration

Objective: find where the autosave flow is implemented.

Role: explorer.

Scope: read-only on `src/` and `tests/`.

Deliverable: list of relevant files, brief flow summary, and risks.

Do not make changes.

If anything is unclear, mark the uncertainty.

Test mapping

Objective: identify existing tests for route validation.

Role: explorer.

Scope: search in `tests/`, `spec/`, and CI scripts.

Deliverable: simple table with file, purpose, and coverage.

Do not edit anything.

Bounded implementation

Objective: add a helper to normalize titles in `src/utils/title.js`.

Role: worker.

Exclusive ownership: only `src/utils/title.js`.

Constraints: do not touch other files, do not change public interfaces.

Deliverable: final code, brief explanation, and verification command.

Document or draft

Objective: write a technical draft on subagent usage.

Role: worker.

Ownership: only `src/02-subagentes-inteligentes.md`.

Style: clear, serious, actionable, in neutral Rioplatense Spanish.

Include criteria, anti-patterns, examples, and a closure checklist.

Do not generate extra files.

Independent verification

Objective: review the change and look for logical errors or scope creep.

Role: explorer.

Scope: read the patch and touched files.

Deliverable: concrete findings, open questions, and validation suggestions.

Do not modify files.

BIBLIA - Nicolás Ezequiel Melluso

21/92

ध्यान दें क़हिर मामले में prompt role, scope, deliverable और limits परभाषति करता है। इससे ambiguity कम होती है और output गुणवत्ता बेहतर होती है।

Closure checklist

डेलीगेशन subagent के ज़वाब देते ही ख़त्म नहीं होता। यह तब ख़त्म होता है जब परणाम verify हो जाए और main thread बना संदेह आगे बढ़ सके।

हमेशा उपयोग करने योग्य closure checklist:

objective एक verify की जा सकने वाली पंक्ति में पूरा हुआ।

subagent ने authorized ownership के भीतर काम किया।

scope से बाहर कोई changes नहीं हुए।

touched files पहचानी गईं।

output reproducible या reviewable है।

code changes होने पर validation command मौजूद है।

document changes होने पर content माँगी गई structure पूरी करता है।

uncertainties स्पष्ट रूप से बताई गईं।

parallel work से कोई conflict नहीं है।

final result को बंद मानने से पहले main thread ने पढ़ लिया।

sensitive tasks के लिए अधिक सख़्त संस्करण:

मैंने diff review किया।

मैंने verify किया कि external files नहीं छुई गईं।

मैंने संबंधित validation command चलाई।

मैंने confirm किया कि hidden assumptions नहीं हैं।

अगर कुछ बाहर रह गया, तो pending items दर्ज़ किए।

Recommended process

subagents उपयोग करने का robust flow आम तौर पर ऐसा दिखता है:

1. task और उसके output criteria परभाषति करें।
2. exploration और execution अलग करें।
3. explicit ownership assign करें।
4. ambiguity और risk के आधार पर model level और effort चुनें।
5. tasks को parallel तभी चलाएँ जब वे overlap न करें।
6. results को main thread में consolidate करें।
7. closure को evidence के साथ verify करें।

अगर task बड़ा है, तो यह flow परतों में दोहराया जा सकता है: पहले explorers, फिर workers, फिर independent review।

BIBLIA - Nicolás Ezequiel Melluso

22/92

अच्छे संकेत

आप सही जा रहे हैं जब:

subagent कम text में अधिक precision देता है;

हर output concrete files या commands बताती है;

main thread बेहतर filtered जानकारी मलिन से तेज नरिणय लेता है;

parallelism समय घटाता है और rework नहीं बढ़ाता;

integration से पहले errors पकड़े जाते हैं।

आप गलत जा रहे हैं जब:

आपको लौटी हर चीज़ फरि से interpret करनी पड़ती है;

surprise changes आते हैं;

context नरियंत्रण से बाहर बढ़ता है;

validation सरिफ अंत में होती है और surprises देती है;

हर subagent को वही base फरि से समझाना पड़ता है।

ऑपरेशनल क्लोजर

subagents का बुद्धिमान उपयोग मात्रा का नहीं, तरीके का प्रश्न है। delegation की गुणवत्ता agents की संख्या से अधिक scoping, ownership और verification पर निर्भर करती है।

अगर exploration चाहिए, explorer उपयोग करें। अगर execution चाहिए, worker उपयोग करें। अगर एक साथ कई काम करने हैं,

काम इस तरह बाँटें कि टिकराव न हो। और अगर आप clear ownership परभाषति नहीं कर सकते, तो अभी डेलीगेट न करें।

सबसे अच्छी प्रैक्टिस "AI को अकेले काम करने देना" नहीं है। बल्कि ऐसी काम-शृंखला बनाना है जहाँ हर हिस्से की भूमिका सीमति हो,

output जाँची जा सके, और closure स्पष्ट हो। तब subagents एक वादे से आगे बढ़कर सच में उपयोगी tool बनते हैं।

BIBLIA - Nicolás Ezequiel Melluso

23/92

खंड 03

SDD और सपोर्ट संरचना

AI-सक्षम repositories में लागू Specification-Driven Development

BIBLIA - Nicolás Ezequiel Melluso

24/92

SDD, यानी Specification-Driven Development, काम करने का ऐसा तरीका है जिसमें specification कोई सजावटी दस्तावेज या अलग-थलग नोट नहीं होती: वही development flow का केंद्रीय हिस्सा होती है। बखिरे हुए code से शुरू करने के बजाय, हम पहले साफ़ परिभाषा से शुरू करते हैं कि क्या बनाना है, क्यों बनाना है, उसे कैसे validate करना है, और रास्ते में कौन-से नरिणय दर्ज होंगे। एक modern repository में, और खासकर जब AI शामिल हो, SDD काम को परतों में व्यवस्थित करता है। पहले समस्या परिभाषित होती है। फिर उसे verifiable specification में बदला जाता है। उसके बाद tasks में तोड़ा जाता है। तभी implementation होता है। और अंत में traceability छोड़ी जाती है: क्या बदला, क्या छोड़ा, क्या test हुआ, और क्या operate करने के लिए तैयार है। यह खासतौर पर तब उपयोगी है जब टीम AI को copilot, draft generator या analysis assistant की तरह इस्तेमाल करती है। AI बहुत तेज़ी ला सकता है, लेकिन assumptions गढ़ भी सकता है, priorities मिला सकता है, या ऐसा code बना सकता है जो “चलता” तो है पर context का सम्मान नहीं करता। SDD इस पर उपयोगी सीमाएँ लगाता है। AI लक्ष्य का अंदाज़ा नहीं लगाता: वह उसे पढ़ता है। वह acceptance criteria तुरंत नहीं गढ़ता: उन्हें follow करता है। वह नरिणयों को replace नहीं करता: उन्हें document करता है या approval के लिए प्रस्तावित करता है।

SDD क्या हल करता है

SDD एक बहुत ठोस समस्या हल करता है: intention और implementation के बीच की दूरी। बड़े repositories में यह दूरी अक्सर चुपचाप बढ़ती रहती है। issue कुछ कहता है, PR कुछ और हल करता है, code आखिर में कुछ तीसरा करता है, और किसी को नहीं पता चलता कि विचार का सही संस्करण कौन-सा था।

SDD के साथ यह शुरुआत स्पष्ट हो जाती है:

requirements : कौन-सी जरूरत मौजूद है और किस उससे फ़र्क पड़ता है।

specs : expected behavior को कैसे वर्णित किया गया है।

decisions : कौन-से विकल्प evaluate हुए और कौन-सा चुना गया।

tasks : कौन-से ठोस कदम execute करने हैं।

acceptance criteria : कैसे पता चलेगा कि काम सही हुआ।

traces : कौन-सा evidence spec को code और tests से जोड़ता है।

runbooks : failure पर इसे कैसे चलाना या recover करना है।

ADRs : context और consequences के साथ architecture decisions।

evaluation notes : validations, assisted testing या comparative analysis के परिणाम।

फायदा सिर्फ़ documentation तक सीमति नहीं है। execution भी बेहतर होता है। जब structure अच्छा हो, AI कम बेकार output देता है, टीम बेहतर चर्चा करती है, और changes review करना आसान हो जाता है।

मूल सिद्धांत

SDD का मुख्य नियम सरल है: हर हिस्से का semantic owner होना चाहिए।

requirement समस्या समझता है।

spec behavior परिभाषित करती है।

decision trade-off समझता है।

task काम को व्यवस्थित करती है।

acceptance scope बंद करती है।

trace धागा ट्रैक करने देती है।

BIBLIA - Nicolás Ezequiel Melluso

25/92

runbook operations के लिए तैयार करती है।

अगर सब कुछ एक ही file में मिला दिया जाए, सिस्टम fragile हो जाता है। अगर बिना मानदंड के बहुत ज्यादा atomize किया जाए, तब भी। SDD संतुलन चाहता है: इतना अलगाव कि हर चीज़ का अपना function रहे, पर इतना नहीं कि एक feature समझने के लिए बीस असंबंधित files खोलनी पड़ें।

सुझाया गया फोल्डर

इस volume के लिए सुझाव है कि SDD material को `.github/orquestador/sdd` के आसपास केंद्रित किया जाए। यह folder specs, decisions और AI-assisted work के governance और execution का core बनता है।

एक संभावित संरचना:

`.github/orquestador/sdd/`

`README.md`

`index.md`

`requirements/`

`000-template.md`

`<area>-<id>.md`

`specs/`

`000-template.md`

`<feature>-<id>.md`

`decisions/`

`000-template-adr.md`

`<adr>-<id>.md`

`tasks/`

`000-template.md`

`<issue>-<id>.md`

`acceptance/`

`000-template.md`

`<feature>-<id>.md`

`traces/`

`000-template.md`

`<feature>-<id>.md`

`runbooks/`

`000-template.md`

`<system>-<id>.md`

`evaluations/`

`000-template.md`

`<feature>-<id>.md`

`examples/`

`sample-issue.md`

`sample-spec.md`

पहले दिन से सभी folders का होना जरूरी नहीं है। महत्वपूर्ण यह है कि architecture scalability के लिए सोचा गया हो और readability बनी रहे।

BIBLIA - Nicolás Ezequiel Melluso

26/92

हर फ़ोल्डर में क्या होगा

README.md

यह entry point है। इसमें बताना चाहिए कि इस repository में SDD क्या है, folder का लक्ष्य क्या है, और इसे कैसे उपयोग करना है। इसमें पूरी theory नहीं, बल्कि system navigate करने के लिए छोटा guide होना चाहिए।

अपेक्षित सामग्री:

folder का purpose;

naming convention;

recommended reading order;

नई spec या नया decision कैसे बनाना है;

issues, PRs और general documentation से संबंध।

index.md

यह navigation map है। इसमें active specs, relevant ADRs, latest evaluations और critical runbooks सूचीबद्ध होते हैं। इसमें status भी हो सकता है: draft, in review, approved, implemented, obsolete।

अपेक्षित सामग्री:

living artifacts का catalog;

cross-links;

document-वार status;

last update date;

owner या area।

requirements/

यहाँ requirements रहती हैं। ये अभी technical tickets नहीं हैं। ये business, product या operations की needs, pains, objectives या constraints हैं।

अच्छी requirement इन सवालों का उत्तर देती है:

कौन-सी समस्या मौजूद है;

कनिके लिए;

अगर हल न हो तो क्या होगा;

कौन-सी constraints हैं;

कौन-से signals success दिखाएँगे।

संक्षिप्त उदाहरण:

BIBLIA - Nicolás Ezequiel Melluso

27/92

REQ-014: ग्राहक onboarding में errors कम करना

समस्या: आज forms अधूरा data save करने देते हैं और इससे rework होता है।

उद्देश्य: persist करने से पहले invalid onboarding रोकना।

सीमाएँ: वर्तमान editing flow नहीं टूटना चाहिए।

प्रभाव: कम manual support और reports में कम errors।

specs/

spec behavior का वर्णन करती है। यह सिर्फ समस्या की बात नहीं करती, बल्कि expected system परभाषित करती है। इसे verifiable होना चाहिए। अच्छी spec inputs, outputs, rules, states, errors, permissions और edge scenarios स्पष्ट करती है। अपेक्षित सामग्री:

context;

scope;

main behavior;

edge cases;

alternate flows;

dependencies;

acceptance criteria;

risks;

explicit assumptions।

spec को code नहीं लिखना चाहिए, पर pseudo-rules, payload examples, state tables या sequences शामिल कर सकती है।

decisions/

यहाँ ADRs और वे structural decisions रहती हैं जिन्हें याद रखना जरूरी है। अगर spec क्या करना है बताती है, तो decision बताता है कि वही रास्ता क्यों चुना गया और दूसरा क्यों नहीं।

अपेक्षित सामग्री:

context;

considered options;

chosen decision;

consequences;

trade-offs;

date;

author या team।

decision का उदाहरण:

BIBLIA - Nicolás Ezequiel Melluso

28/92

ADR-007: validation server पर करें, केवल client पर नहीं

Backend validation को source of truth चुना जाता है।

कारण: client outdated हो सकता है और अकेली barrier के रूप में भरोसेमंद नहीं है।

परिणाम: logic का कुछ भाग duplicate होगा, लेकिन operational risk घटेगा।

tasks/

tasks implementation को छोटे, verifiable steps में तोड़ती हैं। ये spec और code के बीच पुल हैं। अच्छी task कोई vague aspiration नहीं, बल्कि concrete action होती है।

अपेक्षित सामग्री:

task objective;

dependencies;

involved files या modules;

definition of done;

expected evidence।

खराब task कहती है: "feature बना दो।" उपयोगी task कहती है: "endpoint Y में field X की validation जोड़ो और integration test से cover करो।"

acceptance/

यह folder acceptance criteria रखता है। इसका काम request और delivery के बीच contract बंद करना है। यह इतनी concrete हो कि reviewer या AI बिना मनमानी व्याख्या के इसे check कर सके।

अपेक्षित सामग्री:

pass/fail conditions;

nominal scenarios;

expected errors;

observable behavior;

लागू होने पर non-functional requirements।

traces/

traces requirement, spec, tasks और final result को जोड़ती हैं। इससे पता चलता है कि हर decision कहाँ से आया और कौन-सा evidence उसे support करता है।

अपेक्षित सामग्री:

link REQ -> SPEC -> TASK -> PR -> TEST ;

coverage summary;

commits, issues या PRs के references;

जो scope से बाहर रह गया उस पर notes।

BIBLIA - Nicolás Ezequiel Melluso

29/92

runbooks/

runbooks बताती हैं कि failure होने पर flow को कैसे operate या recover करना है। ये खासकर तब उपयोगी हैं जब spec का परिणाम real-impact feature हो: payments, authentication, jobs, synchronizations, migrations या automated processes। अपेक्षित सामग्री:

common symptoms;
diagnostic steps;
recovery steps;
लागू होने पर rollback;
alert signals;
contacts या dependencies।

evaluations/

यहाँ evaluation notes जाती हैं: human tests, AI tests, internal benchmarks, before/after comparisons, risk analysis या quality reviews।

अपेक्षित सामग्री:

क्या evaluate किया गया;
किस method से;
क्या result मिला;
कौन-से defects आए;
उससे कौन-सा decision लिया गया।

examples/

यह friction घटाने के लिए है। templates और concrete examples adoption को बहुत सुधारते हैं, खासकर जब टीम SDD अभी शुरू कर रही हो।

अपेक्षित सामग्री:

sample issue;
sample spec;
sample ADR;
sample task;
sample checklist।

issue या feature का work cycle

स्वस्थ SDD flow PR से शुरू नहीं होता। यह उससे पहले शुरू होता है, जब course correction अभी सस्ता होता है।

1. requirement खोलना

पहले समस्या लिखी जाती है। कवता की जरूरत नहीं, precision की है। requirement को बताना चाहिए कि initiative क्यों है और कौन-सा pain हल करती है।

में

BIBLIA - Nicolás Ezequiel Melluso

30/92

2. इसे spec में बदलना

फरि expected behavior define होता है। यहाँ उत्तर दिया जाता है:

क्या input है;

क्या output है;

कौन-से rules लागू हैं;

कौन-सी errors allowed हैं;

इस version में क्या शामिल नहीं है।

3. decisions दर्ज करना

अगर relevant alternatives हों तो ADR दर्ज करें। इससे दो महीने बाद कोई पहले से तय decision पर फरि वैसी ही बहस नहीं करेगा जैसे कुछ हुआ ही नहीं।

4. tasks में बाँटना

implementation को छोटे tasks में बाँटा जाता है। हर task independently की जा सके, review हो सके और test हो सके।

5. acceptance define करना

code छूने से पहले acceptance criteria लखी होनी चाहिए। अगर बाद में लखी जाए, तो अक्सर वे result के हिसाब से ढल जाती हैं और value खो देती हैं।

6. traceability के साथ implement करना

हर change को spec से connection रहनी चाहिए। यह PR references, task notes या trace links से हो सकता है। महत्वपूर्ण यह है कथागा न टूटे।

7. evaluate करना

अंत में जो हुआ उसे दर्ज करें: कौन-से tests चले, कौन-सी समस्याएँ मलीं, कहाँ coverage कम है, कौन-सा debt बचा, और final decision क्या रहा।

पूर्ण cycle का एक उदाहरण:

1. REQ-014 ग्राहक onboarding में errors पहचानता है।

2. SPEC-014 validations, messages और states परभाषति करता है।

3. ADR-007 backend validation चुनता है।

4. TASK-014.1 validations जोड़ता है।

5. TASK-014.2 integration tests cover करता है।

6. AC-014 5 acceptance criteria परभाषति करता है।

7. TRACE-014 issue, PR और tests को link करता है।

8. EVAL-014 परणाम और open points summarize करता है।

BIBLIA - Nicolás Ezequiel Melluso

31/92

छोटी templates

Requirement

REQ-XXX: छोटा शीर्षक

समस्या:

उद्देश्य:

प्रभावति उपयोगकर्ता:

सीमाएँ:

न करने का जोखिम:

Spec

SPEC-XXX: छोटा शीर्षक

संदर्भ:

दायरा:

दायरे से बाहर:

अपेक्षति व्यवहार:

edge cases:

acceptance criteria:

ADR

ADR-XXX: छोटा नरिणय

संदर्भ:

वकिल्प:

नरिणय:

परणाम:

Task

TASK-XXX: ठोस कार्रवाई

क्या करना है:

संभावति files:

dependencies:

कैसे validate होगा:

BIBLIA - Nicolás Ezequiel Melluso

32/92

Acceptance

AC-XXX: feature या spec

- Given ...

- When ...

- Then ...

Trace

TRACE-XXX: छोटा शीर्षक

REQ:

SPEC:

TASKS:

PR:

TESTS:

NOTAS:

Runbook

RUN-XXX: system या flow

लक्षण:

नदिन:

रकिवरी:

Rollback:

Evaluation notes

EVAL-XXX: छोटा शीर्षक

क्या test किया गया:

मापदंड:

परिणाम:

समस्याएँ:

नरिणय:

AI के साथ SDD कैसे उपयोग करें

AI documentation की value बदल देता है। पहले spec मुख्यतः इंसानों के लिए थी। अब यह assistants के लिए context contract भी है।

BIBLIA - Nicolás Ezequiel Melluso

33/92

AI के साथ एक अच्छा flow यह है:

AI requirement पढ़कर समस्या summarize करे;

AI शुरुआती spec प्रस्तावित करे;

ईसान scope और priorities validate करे;

AI tasks को तोड़े और tests सुझाए;

ईसान decisions approve या correct करे;

AI implementation और traceability review में मदद करे;

ईसान final acceptance verify करे।

यह तब बेहतर चलता है जब repository में artifacts stable और readable हों। spec बखिरी होगी तो AI के जवाब भी बखिरेंगे। acceptance vague होगी तो AI gaps assumptions से भर देगा। ADRs नहीं होंगी तो decisions खो जाएँगे और context हर बार reset होगा।

कुंजी यह है कि AI को reasoning तेज़ करने के लिए उपयोग करें, उसे replace करने के लिए नहीं। SDD AI को explicit आधार पर काम करने देता है। “मेरे लिए feature बना दो” की जगह “इस spec और इन acceptance criteria के आधार पर implementation तोड़ो और risks चिन्हित करो” कहना बेहतर है। भाषा का यह बदलाव output quality को बहुत सुधारता है।

Anti-patterns

सब कुछ एक ही file में मलाना

जब requirements, spec, task और decision बना structure के साथ रहते हैं, document उपयोग के लायक नहीं रहता। किसी को नहीं पता कौन-सा भाग source of truth है।

अस्पष्ट specifications

“यह intuitive होना चाहिए” या “यह अच्छे से चलना चाहिए” जैसी पंक्तियाँ तब तक बेकार हैं जब तक उन्हें verifiable criteria में न बदला जाए। अगर test नहीं हो सकता, तो यह अधूरा है।

बहुत बड़े tasks

“पूरा flow implement करो” जैसी task अक्सर ऐसी PR में बदलती है जहाँ review करना कठिन होता है और traceability debt छोड़ती है।

देर से लिखें ADRs

context भूल जाने के बाद decisions लिखना उनकी value खत्म कर देता है। ADR तभी काम का है जब वह वास्तविक चर्चा को संरक्षित करे।

acceptance को result के हिसाब से फरि लिखना

अगर acceptance criteria code result से मेल कराने के लिए बदली जाती है, तो वे contract नहीं रहती, justification बन जाती है।

खाली traces

सरिफ “PR में है” कहना काफी नहीं। अगर मूल requirement तक धागा नहीं जाता, तो traceability मौजूद नहीं है।

BIBLIA - Nicolás Ezequiel Melluso

34/92

ghost runbooks

ऐसी runbook जस emergency में कोई चला न सके, बेकार है। वह छोटी, स्पष्ट और actionable होनी चाहिए।

नष्टिकर्ष बना evaluations

कुछ मापकर यह न बताना कि उससे क्या decision लिया गया, अधूरा काम है। evaluation को ठोस नष्टिकर्ष छोड़ना चाहिए।

Practical checklist

SDD के तहत feature बंद करने से पहले यह जाँचना उपयोगी है:

requirement लिखी हुई है और उसमें समस्या, उद्देश्य और impact है;

spec verifiable behavior define करती है;

scope में क्या शामिल नहीं है, साफ है;

relevant decisions documented हैं;

tasks छोटे steps में बाँटी गई हैं;

acceptance criteria concrete हैं;

issue, spec, PR और tests के बीच traceability है;

अगर flow operations को प्रभावित करता है, runbook मौजूद है;

evaluation या validation notes दर्ज हैं;

document, code और tests में वरीधाभास नहीं है;

repository oral memory पर निर्भर हुए बिना context reconstruct कर सकती है।

रखरखाव की सफ़िरशि

इस structure को bureaucratic बनने से बचाने के लिए तीन नियम बनाए रखना उपयोगी है:

1. कम लिखो, पर उपयोगी लिखो;

2. code की गतिके साथ ही update करो;

3. हर feature evidence के साथ बंद करो।

मकसद सिर्फ दस्तावेज़ बनाना नहीं है। मकसद है कि documentation काम की infrastructure बने। AI-enabled repo में यह

infrastructure उत्पाद का हिस्सा है। यह errors घटाती है, review बेहतर करती है, और knowledge को context rotation के

बावजूद जीवित रखती है।

समापन

SDD कोई naming trend नहीं है, न engineering का जादुई वकिल्प। यह एक अनुशासन है ताकि code intention को धुंधला करने से

पहले intention व्यवस्थित हो जाए। जनि repositories में AI writing, review या analysis में भाग लेता है, वहाँ यह अनुशासन और

ज़रूरी हो जाता है क्योंकि

ambiguity की लागत कई गुना बढ़ती है।

.github/orquestador/sdd की संरचना एक सरल प्रस्ताव देती है: सही हिस्सों को अलग करो ताकि हर हिस्सा अपना काम करे

और सब मलिकर readable system बनें। Requirements, specs, decisions, tasks, acceptance criteria, traces, runbooks,

BIBLIA - Nicolás Ezequiel Melluso

35/92

ADRs और evaluation notes सजावटी folders नहीं हैं। ये वह mechanism हैं जनिसे वचिर समस्या से implementation तक जाता है और रास्ते में context नहीं खोता।

अगर repository यह काम करने का तरीका अपनाती है, तो हर feature भरोसे की छलांग नहीं रहती। वह एक traceable, reviewable और improvable process बन जाती है। और AI-assisted stack में इसकी कीमत किसी भी तात्कालिक shortcut से ज्यादा है।

BIBLIA - Nicolás Ezequiel Melluso

36/92

खंड 04

AGENTS.md, .github और Prompt-

स्टाइल Commands

agents, SDD, Copilot, workflows और reusable prompts के लिए तैयार repository
कैसे बनाएं

BIBLIA - Nicolás Ezequiel Melluso

37/92

यह खंड क्यों मौजूद है

एक modern repository को इस बात पर निर्भर नहीं होना चाहिए कि इसान हर बार AI assistant खोलते समय सारे नियम याद रखे। प्रोजेक्ट के नियम फाइलों में रहने चाहिए। कुछ नियम इसानों के लिए होते हैं, कुछ agents के लिए, कुछ Copilot के लिए, कुछ CI के लिए, और कुछ SDD प्रोसेस के लिए। जब ये परते मलि जाती हैं, तो AI अधूरे context के साथ काम करता है और टीम बार-बार वही गलतियां सुधारती रहती है।

यह खंड उन repositories के लिए एक practical structure प्रस्तावित करता है जो AI को गंभीरता से उपयोग करना चाहते हैं:

1. AGENTS.md को code agents के operational contract के रूप में।
2. .github/copilot-instructions.md को Copilot के general instructions के रूप में।
3. .github/instructions/*.instructions.md को path-specific instructions के रूप में।
4. .github/prompts/*.prompt.md को reusable prompt-स्टाइल commands के रूप में।
5. .github/orquestador/sdd/* को specifications, decisions और traceability के सिस्टम के रूप में।
6. .github/workflows/* को एकमात्र executable automation layer के रूप में।
7. .github/orquestador/pipelines/catalog.md को governance catalog के रूप में, न कि दूसरी automation engine के रूप में।

उद्देश्य repository को ceremony से भरना नहीं है। उद्देश्य यह है कि हर फाइल का उद्देश्य स्पष्ट हो और कोई agent तीन सवाल जल्दी उत्तर दे सके: यह कौन-सा प्रोजेक्ट है, यहां काम कैसे होता है, और बदलाव सही है यह कैसे verify होता है।

जम्मेदारियों का मानचित्र

फाइल या फोल्डर

मुख्य audience

जम्मेदारी

README.md

नए लोग

प्रोडक्ट, इंस्टॉलेशन और उपयोग समझना

AGENTS.md

code agents

operational rules, commands, permissions, style और

closure

.github/copilot-instructions.md

GitHub Copilot

repository की persistent general instructions

.github/instructions/*.instructions.md

zone-wise

Copilot

specific paths पर लागू नियम

.github/prompts/*.prompt.md

टीम और

assistants

बार-बार होने वाले tasks के invocable prompts

.github/orquestador/context/*

टीम और agents

stable context: product, architecture, glossary

.github/orquestador/sdd/*

टीम और agents

specs, decisions, tasks, traceability, runbooks

.github/orquestador/pipelines/catalog.md

Maintainers

workflows, permissions, risks और owners की

inventory

.github/workflows/*

GitHub Actions

वास्तविक automation: CI, reviewers, validations

BIBLIA - Nicolás Ezequiel Melluso

38/92

Golden rule: context file को execution का दिखावा नहीं करना चाहिए। Workflow execute करता है। Catalog document करता है। Prompt दशा देता है। AGENTS.md agent behavior govern करता है। यह separation बनाए रखने से confusing systems से बचाव होता है।

अनुशसति संरचना

AI, SDD और GitHub-first तरीके से काम करने वाली repo के लिए एक उचित आधार:

BIBLIA - Nicolás Ezequiel Melluso

39/92

repo/
AGENTS.md
README.md
src/
tests/
docs/
.github/
copilot-instructions.md
instructions/
frontend.instructions.md
backend.instructions.md
tests.instructions.md
prompts/
plan-feature.prompt.md
write-spec.prompt.md
review-pr.prompt.md
generate-tests.prompt.md
write-adr.prompt.md
qa-harness.prompt.md
workflows/
ci.yml
pr-reviewer.yml
orquestador/
README.md
context/
product.md
architecture.md
glossary.md
constraints.md
sdd/
README.md
requirements/
specs/
decisions/
tasks/
traces/
evals/
runbooks/
pipelines/
catalog.md
policies/
permissions.md
safety.md

हर प्रोजेक्ट को पहले दनि से सब कुछ नहीं चाहिए। लेकिन mental architecture होना उपयोगी है। एक MVP AGENTS.md , context/product.md , sdd/specs/ , prompts/ और workflows/ci.yml से शुरू हो सकता है। बाकी तब जोड़ा जाए जब वास्तविक repetition देखें।

BIBLIA - Nicolás Ezequiel Melluso

40/92

अच्छा AGENTS.md कैसे लिखें

AGENTS.md वह फाइल है जो agent को बताती है कि repository में operate कैसे करना है। यह landing page नहीं है। यह product vision नहीं है। यह ईंसानों के लिए लंबा manual नहीं है। यह operational guide है।

इसे इन सवालों का जवाब देना चाहिए:

1. Stack क्या है।
2. हर महत्वपूर्ण चीज कहाँ है।
3. install, test, validate और run के लिए कौन-से commands हैं।
4. कौन-सी files या folders sensitive हैं।
5. बदलावों की expected style क्या है।
6. कौन-सी permissions या actions नषिद्ध हैं।
7. task बंद कैसे करना है।

Base example:

BIBLIA - Nicolás Ezequiel Melluso

41/92

AGENTS.md

Project Snapshot

Este repo contiene una app Node.js + TypeScript con API en `src/server`,
frontend en `src/web` y tests en `tests`.

Commands

- Instalar: `npm ci`
- Desarrollo: `npm run dev`
- Tests: `npm test`
- Typecheck: `npm run typecheck`
- Build: `npm run build`

Working Rules

- Mantener cambios acotados al pedido.
- No modificar migraciones antiguas sin pedir permiso.
- No tocar secretos ni archivos `.env`.
- Preferir tests cerca del comportamiento cambiado.
- Si un comando falla, reportar el error exacto y el proximo paso.

SDD

- Specs: `.github/orquestador/sdd/specs/`
- Decisiones: `.github/orquestador/sdd/decisions/`
- Tareas: `.github/orquestador/sdd/tasks/`
- Trazabilidad: `.github/orquestador/sdd/traces/`

Completion

Antes de cerrar, informar:

- Archivos modificados.
- Pruebas ejecutadas.
- Pruebas no ejecutadas y por que.
- Riesgos residuales.

सामान्य गलती यह है कि AGENTS.md बहुत दार्शनिक लिख दिया जाता है। Agent को "best practices अपनाओ" जैसी पंक्तियां नहीं चाहिए। उसे commands, paths, limits और criteria चाहिए।

नरिदेशों की hierarchy

नरिदेशों की precedence होती है। यूजर का explicit request, repo के general rule से ज्यादा वजन रखता है। किसी subfolder के पास वाला AGENTS.md, root AGENTS.md के नियमों को specialize कर सकता है। System या platform instructions सबसे ऊपर priority रखते हैं।

BIBLIA - Nicolás Ezequiel Melluso

इसे practical तरीके से ऐसे समझें:

System / platform

> बातचीत में User

> touched file के सबसे करीब AGENTS.md

> Root AGENTS.md

> सहायक documentation

यह monorepos में खास तौर पर उपयोगी है। Root AGENTS.md global काम का तरीका बता सकता है, जबकि packages/mobile/AGENTS.md mobile-specific commands और conventions तय करता है।

.github/copilot-instructions.md

GitHub, Copilot के लिए repository custom instructions को .github/copilot-instructions.md फाइल में document करता है। इसका काम है Copilot को repository के अंदर काम करते समय persistent context देना।

यह फाइल AGENTS.md से छोटी होनी चाहिए। हर command दोहराने की जरूरत नहीं। यह style, architecture, response preferences और expected validations पर फोकस कर सकती है।

Example:

Copilot Instructions

Este proyecto usa TypeScript estricto. Evitar `any` salvo justificación.

Preferir funciones puras en la capa de dominio.

No crear dependencias nuevas sin explicar el motivo.

Cuando sugieras código, incluir pruebas relevantes.

Si falta contexto de negocio, preguntar o marcar supuesto.

इसे general quality guide करने के लिए उपयोग करें। इसे बहुत बड़ा document न बनाएं। अगर Copilot को बहुत ज्यादा general rules मिलें, तो conflicts या कुछ content ignore होने की संभावना बढ़ती है।

.github/instructions/*.instructions.md

Path-based instructions आपको यह कहने देते हैं: "जब repo के इस हिस्से पर काम करो, ये नियम लागू करो।" GitHub

.github/instructions के अंदर NAME.instructions.md pattern को applyTo frontmatter के साथ document करता है।

Backend example:

BIBLIA - Nicolás Ezequiel Melluso

43/92

```
---
applyTo: "src/server/**/*.ts,tests/server/**/*.ts"
---
```

Backend Instructions

- Validar entradas en la frontera HTTP.
- Mantener reglas de negocio fuera de handlers.
- No acceder a la base de datos desde controllers.
- Agregar tests de casos borde para errores 4xx y 5xx.

Frontend example:

```
---
applyTo: "src/web/**/*.tsx,src/web/**/*.css"
---
```

Frontend Instructions

- Mantener componentes accesibles por teclado.
- Evitar texto que se desborde en mobile.
- Usar componentes existentes antes de crear nuevos.
- No introducir cambios visuales globales sin justificar.

फायदा है precision। frontend की rule backend में फैलने के बजाय, हर zone को वही मलित है जो उसे चाहिए।

.github/prompts/*.prompt.md

Prompt files reusable commands होते हैं। GitHub इन्हें .prompt.md extension वाली Markdown files के रूप में document करता है, आम तौर पर .github/prompts के अंदर। ये उन tasks के लिए हैं जिनमें टीम बार-बार करती है: feature plan करना, PR review करना, tests generate करना, ADR लिखना, API document करना, या onboarding तैयार करना। इन्हें versioned commands की तरह treat करें। अगर कोई prompt बेहतर हो, तो PR में review करें। fail हो तो adjust करें। काम न आए तो remove करें।

plan-feature.prompt.md example:

BIBLIA - Nicolás Ezequiel Melluso

44/92

description: "Convertir una idea de producto en plan SDD por slices"

Usa el contexto de:

- [producto](../orquestador/context/product.md)
- [arquitectura](../orquestador/context/architecture.md)

Entrada del usuario:

\${input:feature:Describe la feature}

Producir:

1. Problema y objetivo.
 2. No objetivos.
 3. Supuestos.
 4. Requisitos funcionales.
 5. Criterios de aceptacion.
 6. Slices de implementacion.
 7. Tests recomendados.
 8. Riesgos y preguntas abiertas.
- No inventes reglas de negocio. Marca lo desconocido.
- review-pr.prompt.md example:

description: "Revisar un PR con foco en bugs, riesgos y pruebas"

Revisa los cambios del PR actual.

Prioridad:

1. Bugs o regresiones.
2. Riesgos de seguridad o permisos.
3. Falta de tests para comportamiento nuevo.
4. Inconsistencias con SDD o ADRs.

Salida:

- Findings con archivo y linea cuando sea posible.
- Preguntas abiertas.
- Resumen breve.

No hagas comentarios de estilo si no afectan mantenimiento o comportamiento.

qa-harness.prompt.md example:

BIBLIA - Nicolás Ezequiel Melluso

45/92

description: "Disenar un harness de evaluacion para una capacidad de IA"

Capacidad a evaluar:

\$(input:capability:Describe la capacidad)

Disena:

1. Fixtures de entrada.
2. Salidas esperadas o criterios.
3. Rubrica de scoring.
4. Casos negativos.
5. Script CLI minimo.
6. Como integrarlo en CI.
7. Riesgos de falsos positivos.

ये फाइलें टीम के ज्ञान को invocable बनाती हैं। ये human judgment को replace नहीं करती, लेकिन variability कम करती हैं।

.github/orquestador

.github/orquestador फोल्डर काम करने के सिस्टम का home बन सकता है। नाम जादुई नहीं है। महत्वपूर्ण यह है कि इसकी ज़िम्मेदारी स्पष्ट हो: context, SDD, policies और catalogs को एक जगह लाना ताकि humans और agents को दिशा मिले।

एक उपयोगी convention:

.github/orquestador/

README.md Repo operating system का index

context/ Stable context

sdd/ Specifications और traceability

prompts/ Internal prompts, अगर '.github/prompts' use न हो

pipelines/ Workflows और automations का catalog

policies/ Permissions, security, risk criteria

अगर टीम GitHub Copilot बहुत उपयोग करती है, तो tool compatibility के लिए .github/prompts बनाए रखना बेहतर है और

.github/orquestador को context और SDD के लिए रखें। अगर कोई दूसरा agent उपयोग होता है, तो

orquestador/prompts भी हो सकता है। महत्वपूर्ण है बिना ज़रूरत duplication न करना।

Repo के अंदर SDD

SDD का अर्थ है specifications से काम करना, impulses से नहीं। AI वाली repository में SDD का critical role है: यह agent को request की creative interpretation implement करने से रोकता है।

एक minimal spec में यह शामिल होना चाहिए:

1. Problem.

BIBLIA - Nicolás Ezequiel Melluso

46/92

2. Objective.
3. Non-objectives.
4. Requirements.
5. Acceptance criteria.
6. Edge cases.
7. Technical impact.
8. Slice-wise plan.
9. Tests.
10. Issue, PR और decisions के साथ traceability.

Path example:

.github/orquestador/sdd/specs/2026-05-08-reassignar-reclamo.md

Header example:

Reassignar reclamo a otro equipo

Estado: Draft

Owner: Producto / Backend

Issue: #123

PRs: pendiente

Problema

Los operadores no pueden mover un reclamo cuando fue derivado al equipo incorrecto.

Criterios de aceptacion

- Un operador autorizado puede reassignar el reclamo.
- La reasignacion queda auditada.
- El equipo anterior y el nuevo quedan visibles en el historial.
- Si el reclamo esta cerrado, no puede reasignarse.

जब agent implement करे, उसे इस spec से स्पष्ट समझ आना चाहिए कि "done" का मतलब क्या है।

Workflows को एकमात्र executable layer रखना

अगर repository में GitHub मुख्य surface है, तो .github/workflows को एकमात्र executable automation layer रखना बेहतर है। बाकी चीजें document, guide या govern कर सकती हैं। यह separation दो समस्याएं रोकता है: duplicate automations और यह भ्रम कि execution वास्तव में कहां हो रहा है।

शुरुआत के लिए conservative pattern:

BIBLIA - Nicolás Ezequiel Melluso

47/92

```
name: Safe PR Reviewer
on:
  pull_request:
    types: [opened, synchronize, reopened]
permissions:
  contents: read
  pull-requests: read
  issues: write
concurrency:
  group: pr-reviewer-${{ github.event.pull_request.number }}
  cancel-in-progress: true
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - name: Comment with checklist
        uses: actions/github-script@v7
        with:
          script: |
            const body = [
              "Revision automatica inicial:",
              "- Verificar tests.",
              "- Confirmar criterios SDD.",
              "- Revisar permisos y secretos.",
              "- Mantener cambios acotados."
            ].join("\n");
            await github.rest.issues.createComment({
              owner: context.repo.owner,
              repo: context.repo.repo,
              issue_number: context.payload.pull_request.number,
              body
            });
```

यह workflow checkout नहीं करता, PR code execute नहीं करता, और conservative तरीके से comment करता है। यह पहला अच्छा automation है क्योंकि

यह बना बड़ा risk surface खोले व्यवस्था देता है।

Pipeline catalog

Catalog execute नहीं करता। यह document करता है। इसे यह जवाब देना चाहिए:

1. कौन-सा workflow मौजूद है।
2. उसे कौन-सा event trigger करता है।

BIBLIA - Nicolás Ezequiel Melluso

48/92

3. वह कौन-सी permissions उपयोग करता है।
4. उसके क्या risks हैं।
5. उसका maintainer कौन है।
6. वह कौन-सा output देता है।
7. उसे क्या करने की अनुमति नहीं है।

Example:

```
# Pipeline Catalog
## safe-pr-reviewer
- Archivo: `.github/workflows/pr-reviewer.yml`
- Evento: `pull_request`
- Permisos: `contents: read`, `pull-requests: read`, `issues: write`
- Ejecuta código del PR: no
- Output: comentario con checklist
- Owner: Platform
- Riesgo principal: ruido en PRs si las reglas no se mantienen
- Estado: activo
```

जब कोई पूछे "इस repo में कौन-सी automations हैं?", तो catalog जवाब देता है। जब कोई पूछे "असल में क्या execute होता है?", जवाब फिर भी .github/workflows ही है।

हमसे आपस में कैसे जुड़ते हैं

एक complete flow यह हो सकता है:

1. इंसान एक issue बनाता है।
2. वह plan-feature.prompt.md चलाकर idea को spec में बदलता है।
3. spec को .github/orquestador/sdd/specs/ में सेव करता है।
4. agent AGENTS.md , context और spec पढ़ता है।
5. agent एक scoped slice implement करता है।
6. वह AGENTS.md में बताए tests चलाता है।
7. वह PR खोलता है।
8. pr-reviewer.yml checklist comment करता है।
9. दूसरा agent या इंसान review-pr.prompt.md उपयोग करता है।
10. SDD में traceability अपडेट की जाती है।

महत्वपूर्ण बात यह है कि हर कदम traces छोड़ता है। टीम को chat में क्या बात हुई, यह याद रखने पर निर्भर नहीं रहना पड़ता।

BIBLIA - Nicolás Ezequiel Melluso

49/92

सामान्य गलतियाँ

गलती

परिणाम

सुधार

बहुत बड़ा AGENTS.md

agent कुछ हिससे ignore करता है

इसे operational रखें और लंबे docs को link करें

पाँच फाइलों में नियम दोहराना

conflicting instructions

layer-wise owner तय करें

unversioned prompts

हर व्यक्ति अलग variant उपयोग करता

है

इन्हें .github/prompts में रखें

broad permissions वाले

workflows

अनावश्यक risk

प्रति workflow minimum permissions

execution का वादा करता catalog

operational confusion

catalog document करता है, workflows execute करते

हैं

acceptance criteria बना specs

ambiguous implementations

हर spec को tests और examples से close करें

Bootstrap checklist

repo को तैयार छोड़ने के लिए:

1. वास्तविक commands के साथ AGENTS.md बनाएं।
2. छोटा .github/copilot-instructions.md बनाएं।
3. .github/instructions/ तभी बनाएं जब path-wise rules हों।
4. 3 से 5 उपयोगी prompts के साथ .github/prompts/ बनाएं।
5. .github/orquestador/context/product.md बनाएं।
6. .github/orquestador/context/architecture.md बनाएं।
7. .github/orquestador/sdd/README.md बनाएं।
8. specs/ , decisions/ , tasks/ , traces/ , evals/ , runbooks/ बनाएं।
9. .github/orquestador/pipelines/catalog.md बनाएं।
10. verify करें कि .github/workflows ही automation execute करने वाली एकमात्र layer है।
11. एक test PR बनाकर पुष्टि करें कि instructions समझने योग्य हैं।

सत्यापति स्रोत

GitHub, Copilot के लिए repository instructions document करता है, जिनमें .github/copilot-instructions.md , .github/instructions/*.instructions.md , और agents instructions के लिए AGENTS.md का उपयोग शामिल है:

GitHub Docs - Adding repository custom instructions for GitHub Copilot

GitHub Docs - Prompt files

GitHub Docs - Your first prompt file

BIBLIA - Nicolás Ezequiel Melluso

50/92

openai/agents.md

नोट: 2026-05-08 को इन स्रोतों की जांच के समय Copilot prompt files को public preview के रूप में document किया गया था, इसलिए इन्हें किसी enterprise में rigid standard बनाने से पहले official documentation दोबारा देखना उचित है।

समापन

लक्ष्य एक प्रभावशाली .github फोल्डर बनाना नहीं है। लक्ष्य यह है कि repo में आने वाला हर agent कम guesswork, ज्यादा verification और बेहतर memory के साथ काम कर सके। AGENTS.md operations के नियम देता है। SDD product और technical contract देता है। Prompt files दोहराए जाने वाले tasks को commands में बदलते हैं। Workflows validations चलाते हैं। Catalog सिस्टम समझाता है।

जब ये हिससे align होते हैं, तो AI एक loose chat नहीं रहता, बल्कि work infrastructure बन जाता है।

BIBLIA - Nicolás Ezequiel Melluso

51/92

खंड 05

Prompt Engineering और Harness

Engineering

अलग-अलग prompts से versioned, evaluable और production-ready systems तक

BIBLIA - Nicolás Ezequiel Melluso

52/92

एक उपयोगी experiment और एक भरोसेमंद system के बीच का फर्क सिर्फ अच्छा prompt लिखने में नहीं है। एक isolated prompt किसी एक काम को हल कर सकता है, लेकिन एक गंभीर product को और चीजें चाहिए: versions, test cases, evaluation criteria, observability, और security rules। यही पूरा सेट किसी idea को operable capability में बदलता है।

यह वॉल्यूम एक सरल विचार से शुरू होता है: prompt को app, notebook या comment में चपिकी हुई loose string की तरह नहीं रहना चाहिए। अगर कोई instruction business के लिए महत्वपूर्ण है, तो उसे auditable, comparable, testable, और disciplined तरीके से deployable होना चाहिए। यहीं harness engineering आती है: उस environment का design जो prompt को execute, measure, और control करता है।

प्रैक्टिस का फोकस बदल जाता है। सवाल सिर्फ "model को क्या कहें?" नहीं रहता, बल्कि "इस task को हर बार same criteria के साथ कैसे चलाऊं, verify कैसे करूं, और system बढ़ने पर इसे टूटने से कैसे बचाऊं?" बन जाता है। यही artisanal prompt engineering से productive prompt engineering की transition है।

जब prompt production में जाता है तो क्या बदलता है

demo में prompt वही text हो सकता है जिसने आज अच्छा result दिया। production में वही text पर्याप्त नहीं रहता क्योंकि नई

conditions आ जाती हैं:

1. model बदलता है।
2. temperature बदलता है।
3. user context बदलता है।
4. input data बदलता है।
5. system maintain करने वाली team बदलती है।
6. legal, security, या brand requirements जुड़ती हैं।

ऐसा होने पर समस्या सिर्फ "response quality" नहीं होती। असली समस्या control की होती है। आपको पता होना चाहिए कि कौन-सी instruction इस्तेमाल हुई, किस version के साथ, कनि inputs पर, कनि tools के साथ, और behavior अभी भी expected सीमा में है या नहीं।

इसीलिए production prompt system में आमतौर पर ये हसिसे होते हैं:

persistent instructions, जो identity और stable behavior define करें;

task prompts, जो specific request solve करें;

fixtures, जो well-defined input cases represent करें;

golden tests, जो expected output या observable properties fix करें;

evals, जो rubric के साथ quality मापें;

regression tests, जो degradation पकड़ें;

permissions और security rules;

context में result review के लिए logging और observability।

ये decorative layers नहीं हैं। assistant, classifier, writer, router, या agent पर भरोसा करने के लिए यही minimum infrastructure है।

BIBLIA - Nicolás Ezequiel Melluso

53/92

एक अच्छे prompt की anatomy

अच्छा prompt सिर्फ लंबा या छोटा होने से तय नहीं होता। वह तब उपयोगी होता है जब ambiguity घटाए, priorities क्रम में रखे, और data missing होने पर क्या करना है यह स्पष्ट करे। प्रैक्टिस में best prompts की structure मिलती-जुलती होती है।

1. स्पष्ट objective

model को पता होना चाहिए कि वह कौन-सी problem solve कर रहा है। सिर्फ "user की मदद करो" काफी नहीं है। objective को operational terms में लिखना बेहतर है।

उदाहरण:

तुम्हारा काम informal requests को clear, actionable और complete work tickets में बदलना है।

2. operation context

prompt को बताना चाहिए कि वह किस environment में इस्तेमाल हो रहा है और उसकी limits क्या हैं। support, sales agent, और internal classifier के लिए writing समान नहीं होती।

उदाहरण:

यह assistant operations team में इस्तेमाल होता है। clarity, traceability, और professional language को priority दो।

3. quality criteria

अगर आप quality define नहीं करते, model खुद बना लेता है। अच्छा prompt यह स्पष्ट करता है कि क्या value है: precision, concision, coverage, tone, safety, format।

उदाहरण:

response concrete होनी चाहिए, data invent नहीं करना चाहिए, और observable facts को assumptions से अलग करना चाहिए।

4. constraints

constraints comfortable लेकिन useless responses से बचाते हैं। इसमें format, length, available tools, languages, exclusions, और security rules आते हैं।

उदाहरण:

अगर जानकारी uncertain हो तो tables का उपयोग न करो। permissions assume न करो। confirmation के बिना actions execute न करो।

5. uncertainty policy

mature prompt बताता है कि context missing होने पर क्या करना है। इससे hallucinations और shaky responses कम होते हैं।

BIBLIA - Nicolás Ezequiel Melluso

54/92

उदाहरण:

अगर critical data missing है तो सिर्फ एक clarification question पूछो। अगर data non-critical है तो explicit assumption के साथ आगे बढ़ो।

6. output format

output इंसान या system की अगली layer दोनों के लिए आसान होनी चाहिए। JSON चाहिए तो schema बताओ। bullets चाहिए तो साफ कहो। template चाहिए तो define करो।

उदाहरण:

हमेशा लौटाओ:

1. Summary
2. Risks
3. Next steps
7. examples

examples सजावट नहीं हैं; वे semantic anchors हैं। वे tone, detail level, और boundary decisions को stabilize करते हैं।

खासकर तब उपयोगी होते हैं जब task में format या criteria ambiguity हो।

एक अच्छा prompt आमतौर पर instructions के साथ 1 या 2 compact examples रखता है। इसे overload नहीं करना चाहिए: बहुत ज्यादा examples noise बढ़ाते हैं।

persistent instructions बनाम task prompts

एक common गलती है सब कुछ एक ही instruction में मिला देना। इससे system fragile हो जाता है। उपयोगी split यह है:

persistent instructions: जो tasks के बीच नहीं बदलना चाहिए;

task prompts: जो हर request में बदलते हैं;

dynamic context: user data, files, session state, tools, temporary memory।

persistent instructions

यह identity और policy layer है। यह role, tone, criteria priority, safety limits, language, और expected behavior define करती है।

उदाहरण:

तुम एक operations assistant हो। तुम neutral Rioplatense Spanish में clear और serious style में जवाब देते हो। तुम

precision, safety और actionable steps को prioritize करते हो।

task prompts

ये किसी specific execution के लिए point instructions हैं। इन्हें short, specific, और result-focused होना चाहिए।

BIBLIA - Nicolás Ezequiel Melluso

55/92

उदाहरण:

इस text के आधार पर अधिकतम 180 शब्दों का executive summary लिखो और अंत में 3 concrete risks दो।

practical rule

अगर कोई instruction हर case में repeat होती है, तो वह persistent layer में जाती है। अगर वह हर execution में बदलती है, तो task layer में जाती है। अगर वह user input में है, तो उसे policy के साथ mix मत करो। इस separation से errors कम होते हैं और system maintainable बनता है।

Fixtures: behavior test करने के लिए concrete cases

fixture एक controlled input case है जो realistic situation को represent करता है। prompt engineering में fixtures जरूरी हैं क्योंकि

ये दिखाते हैं कि system typical, rare, और risky scenarios में कैसा behave करता है।

सिर्फ एक happy example पर्याप्त नहीं है। serious harness को ऐसे fixtures चाहिए जो cover करें:

clean inputs;

incomplete inputs;

contradictory inputs;

noisy inputs;

ambiguous requests;

unsafe output force करने की कोशिशें;

format edge cases।

fixture उदाहरण

```
{
  "id": "ticket-001",
  "input": "मुझे किसी से पछिले महीने की invoice चेक करवानी है क्योंकि
  मुझे लगता है कि वह गलत आई है।",
  "expected_traits": [
    "identifier missing हो तो clarification मांगें",
    "data invent न करें",
    "professional tone बनाए रखें",
    "next step propose करें"
  ]
}
```

अच्छा fixture कसि कहें

अच्छा fixture tricks से model को "हराने" की कोशिश नहीं करता। वह useful situation बताता है। वह होना चाहिए:

stable;

reproducible;

readable;

BIBLIA - Nicolás Ezequiel Melluso

56/92

representative;
easy to extend।

अगर fixture हर समय बदलता रहे तो version comparison नहीं हो सकता। अगर बहुत artificial हो तो real usage reflect नहीं करेगा। quality balance में है।

Golden tests: expected behavior को fix करना

golden tests expected output या बहुत specific output properties के against tests हैं। ये prompt, model, या tool chain बदलने पर regressions detect करते हैं।

दो सामान्य तरीके:

exact golden

जब output बहुत stable होना चाहिए, जैसे structured format।

उदाहरण:

```
{  
  "id": "routing-01",  
  "expected": {  
    "category": "facturacion",  
    "confidence": "alta"  
  }  
}
```

property-based golden

जब full text freeze नहीं करना चाहते, लेकिन behavior freeze करना चाहते हैं।

उदाहरण:

response data invent न करे;

warning शामिल हो;

exactly 3 steps लौटाए;

expected language का उपयोग करे;

unauthorized tools mention न करे।

यह दूसरा format ज्यादा flexible है और natural language systems में अक्सर ज्यादा useful होता है। gold हमेशा exact string नहीं होता; कई बार वह observable rules का compliance होता है।

Harness CLI: चलाओ, compare करो, repeat करो

harness वह environment है जो fixtures के साथ prompts चलाता है, results log करता है, और उन्हें reference के खिलाफ compare करता है। एक अच्छी CLI इसे terminal और CI से repeatable बनाती है।

BIBLIA - Nicolás Ezequiel Melluso

57/92

एक reasonable structure इन चीजों को अलग रखता है:

```
prompt definition;
fixture definition;
model configuration;
batch execution;
evaluation;
report;
result export
संभावित structure
prompt-harness/
prompts/
system.md
task.md
fixtures/
inbox.jsonl
safety.jsonl
formatting.jsonl
evals/
rubric.md
scoring.ts
runs/
2026-05-08T10-30-00Z.jsonl
reports/
latest.md
src/
cli.ts
harness.ts
loader.ts
evaluator.ts
उदाहरण commands
node src/cli.js run --prompt prompts/system.md --fixtures fixtures/inbox.jsonl
node src/cli.js eval --run runs/latest.jsonl --rubric evals/rubric.md
node src/cli.js compare --baseline runs/baseline.jsonl --candidate runs/latest.jsonl
node src/cli.js report --input runs/latest.jsonl --output reports/latest.md
CLI को क्या करना चाहिए
उपयोगी CLI सिर्फ model call नहीं करती। वह:
files के अस्तित्व की validation करती है;
BIBLIA - Nicolás Ezequiel Melluso
58/92
```

inputs normalize करती है;
prompt और model version log करती है;
timestamps save करती है;
raw responses serialize करती है;
metrics compute करती है;
CI के लिए clear summary देती है।
अगर harness सिर्फ सुंदर text print करे, तो वह demo है। अगर traceability और comparison भी दे, तभी वह infrastructure बनना शुरू करता है।
Evals और rubrics: "मुझे अच्छा लगा" से आगे
prompt evaluation सिर्फ intuition पर निर्भर नहीं हो सकता। आपको repeatable criteria चाहिए। rubric यही देता है: "good" का explicit definition।
simple rubric
rubric इन dimensions पर score दे सकता है:
context fidelity;
completeness;
accuracy;
format;
tone;
safety;
actionable usefulness।
उदाहरण:
प्रति dimension score 0-2:
0 = serious failure
1 = partial
2 = meets criteria
BIBLIA - Nicolás Ezequiel Melluso
59/92

evaluation उदाहरण

Dimension

Criterion

Fidelity

data invent नहीं करता और input से contradict नहीं करता

Format

मांगी गई structure का पालन करता है

Tone

professional language बनाए रखता है

Actionability

useful next steps देता है

Safety

forbidden actions execute या recommend नहीं करता

automatic और human evaluation

दोनों approach का संयोजन बेहतर है।

automatic: objective rules, format, length, field presence, prohibited patterns के लिए;

human: quality nuance, clarity, persuasion, usefulness, alignment के लिए।

real systems में common error है weak automatic metrics को full solution मान लेना। वे useful हैं, पर critical reading का replacement नहीं। अच्छी प्रैक्टिस है filtering के लिए automation और decision के लिए human rubric।

Regression tests: जो पहले काम कर रहा था उसे टूटने मत दो

regression test versions के बीच behavior compare करता है। लक्ष्य system को हमेशा के लिए freeze करना नहीं, बल्कि unwanted changes detect करना है।

mature flow में हर prompt या model change को यह जवाब देना चाहिए:

1. क्या बेहतर हुआ;
2. क्या खराब हुआ;
3. कौन-से नए cases देखे;
4. कौन-से trade-off स्वीकार हैं;
5. कहाँ rollback चाहिए।

typical regression cases

नया prompt जरूरत से ज्यादा verbose हो जाता है;

safety warning गायब हो जाती है;

model clarification मांगना बंद कर देता है;

language बदल जाती है;

JSON format parser तोड़ देता है;

tool तब invoke होता है जब नहीं होना चाहिए।

BIBLIA - Nicolás Ezequiel Melluso

60/92

practical strategy

critical fixtures का छोटा set रखें और exploratory fixtures का बड़ा set रखें। critical set essentials बचाता है; exploratory set main path के बाहर behavior दिखाता है।

अगर regression test fail हो, तो सिर्फ "output ठीक करो" काफी नहीं। समझना होगा कि issue कहाँ है:

prompt;

model;

post-processing;

configuration;

policy;

data set।

Security और permissions

जब model actions produce करता है, सिर्फ अच्छा response काफी नहीं। उसे clear limits के भीतर operate भी करना होगा।

tool-enabled systems में security prompt और harness design का हिस्सा है।

basic principles

least privilege;

sensitive actions के लिए confirmation;

suggest और execute का separation;

input और output validation;

decision logging;

dangerous actions का explicit blocking।

rule उदाहरण

अगर कोई action data modify करे, ऐसे charge करे, जानकारी delete करे, या external messages भेजे,

तो execute करने से पहले human confirmation लेना जरूरी है।

secure prompt

prompt को permissions या credentials assume नहीं करने चाहिए। उसे model को यह भी encourage नहीं करना चाहिए कि वह

external validation वाली चीजें "अपने आप solve" कर ले।

उदाहरण:

यह assume मत करो कि तुम्हें external systems का access है। अगर impactful action चाहिए, तो उसे describe करो और

confirmation मांगो।

BIBLIA - Nicolás Ezequiel Melluso

61/92

secure harness

harness को permissions simulate करने और limits test करने में सक्षम होना चाहिए:

बना internet access;

disabled tools के साथ;

fake credentials के साथ;

read-only mode में;

confirmation required mode में।

इससे verify होता है कि system सिर्फ full-access mode में नहीं, बल्कि restricted conditions में भी सही चलता है।

Observability: सच में क्या हुआ, यह देख पाना

observability वह क्षमता है जो system के lab से बाहर जाने के बाद debugging और learning संभव बनाती है। अगर production में prompt fail होता है, तो context reconstruct करना जरूरी होता है।

क्या log करना चाहिए

prompt version;

model version;

date और time;

summarized input;

raw output;

tools used;

latency;

estimated cost;

errors;

policy decision;

fixture या case identifier।

log उदाहरण

```
{
  "run_id": "2026-05-08T10:30:00Z",
  "prompt_version": "1.4.2",
  "model": "gpt-5.3",
  "fixture_id": "safety-03",
  "latency_ms": 1840,
  "tools_used": ["search"],
  "outcome": "needs_review"
}
```

~

~

~

BIBLIA - Nicolás Ezequiel Melluso

62/92

पहले क्या देखना चाहिए

जब कुछ fail हो, तो यह क्रम उपयोगी है:

1. original input;
2. actual applied prompt;
3. model configuration;
4. available tools;
5. raw output;
6. post-processing;
7. evaluation criteria

observability के बिना हर bug जादू जैसा लगता है। observability के साथ वह auditable decisions की sequence बन जाता है।

एक minimal harness उदाहरण

idea को जमीन पर लाने के लिए मान लें कि system internal requests classify करता है और short plan देता है। minimum architecture कुछ ऐसा हो सकता है:

1. persistent instructions load करो
2. task prompt load करो
3. fixture load करो
4. final message बनाओ
5. model चलाओ
6. format validate करो
7. rubric से score करो
8. result save करो
9. baseline से compare करो
10. report करो

pseudocode

```
const system = loadFile("prompts/system.md")
const task = loadFile("prompts/task.md")
const fixtures = loadJsonl("fixtures/inbox.jsonl")
for (const fixture of fixtures) {
  const input = buildInput(system, task, fixture)
  const output = await model.run(input)
  const score = evaluate(output, fixture.expected_traits)
  saveRun({ fixtureId: fixture.id, output, score })
}
```

बि

BIBLIA - Nicolás Ezequiel Melluso

63/92

व्यावहारिक नियम

यदि कोई निर्देश हर मामले में दोहराया जाता है, तो वह शायद persistent layer में जाता है। यदि वह हर execution में बदलता है, तो वह task layer में जाता है। यदि वह user input में रहता है, तो उसे policy के साथ मत मिलाओ। इन चीजों को अलग रखने से errors घटते हैं और system maintainable बनता है।

Fixtures: behavior test करने के ठोस cases

fixture एक controlled input case है जो realistic situation को represent करता है। prompt engineering में fixtures जरूरी हैं, क्योंकि वे दिखाते हैं कि system typical, rare या dangerous scenarios में सही respond करता है या नहीं।

सिर्फ एक happy example काफी नहीं है। गंभीर harness को ऐसे fixtures चाहिए जो cover करें:

clean inputs;

incomplete inputs;

contradictory inputs;

noisy inputs;

ambiguous requests;

unsafe output force करने की कोशिशें;

format edge cases.

fixture example

```
{
  "id": "ticket-001",
  "input": "मुझे चाहिए कि कोई पछिले महीने की invoice review करे, क्योंकि लगता है कि उसमें गलती है।",
  "expected_traits": [
    "identifier missing हो तो clarification मांगें",
    "data invent न करें",
    "professional tone रखें",
    "next step propose करें"
  ]
}
```

अच्छे fixture की पहचान

अच्छा fixture tricks से model को हराने की कोशिश नहीं करता। वह useful situation describe करता है। उसे होना चाहिए:

stable;

reproducible;

legible;

representative;

extend करना आसान.

यदि fixture लगातार बदलता है, तो versions compare करने में काम नहीं आता। यदि वह बहुत artificial है, तो real use को reflect नहीं करता। quality balance में है।

Golden tests: expected behavior को स्थिर करना

Golden tests expected output या output की बहुत concrete properties के against tests हैं। जब prompt, model या tool chain बदलती है, तब ये regressions पकड़ने में मदद करते हैं।

दो common forms हैं:

Exact golden

जब output बहुत stable होना चाहिए, जैसे structured format, तब उपयोगी है।

Example:

```
{
  "id": "routing-01",
  "expected": {
    "category": "billing",
    "confidence": "high"
  }
}
```

Property-based golden

जब पूरा text freeze नहीं करना है, लेकिन behavior freeze करना है, तब उपयोगी है।

Example:

response कोई data invent न करे;

warning include करे;

exactly 3 steps लौटाए;

expected language use करे;

unauthorized tools mention न करे.

दूसरा format ज्यादा flexible है और natural-language systems में अक्सर ज्यादा useful होता है। golden हमेशा exact string नहीं होता; कई बार वह observable rules का पालन होता है।

Harness CLI: run, compare, repeat

harness वह environment है जो prompts को fixtures के साथ execute करता है, results record करता है और reference के against compare करता है। अच्छी तरह design की गई CLI इसे terminal और CI से repeatable बनाती है।

एक reasonable structure अलग रखता है:

prompt definition;

fixture definition;

model configuration;

batch execution;

evaluation;

```
report;
results export.
संभावति structure
prompt-harness/
  prompts/
    system.md
    task.md
  fixtures/
    inbox.jsonl
    safety.jsonl
    formatting.jsonl
  evals/
    rubric.md
    scoring.ts
  runs/
    run-001.jsonl
  reports/
    latest.md
  src/
    cli.ts
    harness.ts
    loader.ts
    evaluator.ts
```

Example commands

```
node src/cli.js run --prompt prompts/system.md --fixtures fixtures/inbox.jsonl
node src/cli.js eval --run runs/latest.jsonl --rubric evals/rubric.md
node src/cli.js compare --baseline runs/baseline.jsonl --candidate runs/latest.jsonl
node src/cli.js report --input runs/latest.jsonl --output reports/latest.md
```

CLI को क्या करना चाहिए

useful CLI सिर्फ model call नहीं करती। वह यह भी करती है:

files exist हैं या नहीं validate करती है;

inputs normalize करती है;

prompt और model version record करती है;

timestamps save करती है;

raw responses serialize करती है;

metrics calculate करती है;

CI के लिए clear summary emit करती है।

यदि harness सिर्फ सुंदर text print करता है, तो वह demo है। यदि वह traceability और comparison भी छोड़ता है, तो वह infrastructure बनने लगता है।

init.sh health contract के रूप में

development harness में सबसे important script हमेशा सबसे sophisticated नहीं होती। कई बार वह एक ही command होती है:

./init.sh

इस script को एक simple question का जवाब देना चाहिए: "क्या यह repo आगे बढ़ने की condition में है?" इसके लिए यह कर सकता है:

1. dependencies मौजूद हैं या नहीं validate करना।
2. tests चलाना।
3. linters या typechecks चलाना।
4. specs की structure validate करना।
5. confirm करना कि एक से अधिक feature in_progress न हों।
6. requirements और tests के बीच traceability missing हो तो fail करना।

Operational rule:

यदि ./init.sh red है, तो feature done नहीं हो सकती।

लक्ष्य CI को replace करना नहीं है। लक्ष्य agent को clear local gate देना है। यद्विह closure "./init.sh execute किया, result green" पर खत्म होता है, तो team कम ambiguity के साथ review कर सकती है।

Recommended minimal contract:

0 = सब green

1 = validation या tests failure

2 = local configuration missing

3 = invalid SDD structure

4 = insufficient permissions या blocked action

यह भी बेहतर है कि script idempotent हो: उसे दो बार चलाने से repo change न हो और invisible state पर depend न करे। यद्विह बहुत slow है, तो quick checks और heavy suites अलग करो और दोनों commands document करो। agent को reliable local gate चाहिए, unpredictable ritual नहीं।

Minimal example:

```
#!/usr/bin/env sh
```

```
set -eu
```

```
command -v node >/dev/null 2>&1 || exit 2
```

```
npm test || exit 1
```

```
npm run lint || exit 1
```

```
node scripts/validate-sdd.mjs || exit 3
```

```
node scripts/check-permissions.mjs || exit 4
```

यदि team मुख्यतः Windows पर काम करती है, तो equivalent init.ps1 या node scripts/init.mjs हो सकता है। जरूरी बात यह है कि documented, repeatable closure command हो और exit codes clear हों।

Requirement-test traceability

गंभीर harness सिर्फ "tests हैं" से संतुष्ट नहीं होता। उसे जानना होता है कि कौन-सा test कौन-से requirement को cover करता है।

Example:

R1 -> test_recent_default_limit

R2 -> test_recent_invalid_limit

R3 -> test_recent_empty_store

Traceability यहाँ रह सकती है:

specs/<feature>/requirements.md ;

specs/<feature>/tasks.md ;

progress/review_<feature>.md ;

harness द्वारा generated table.

Reviewer को reject करना चाहिए यदि:

1. कोई R<n> test के बना है।
 2. नया test किसी requirement से linked नहीं है।
 3. task कहती है कि वह requirement cover करती है, लेकिन test यह prove नहीं करता।
 4. code ऐसा behavior implement करता है जो spec में नहीं है।
- यह common trap रोकता है: tests code exercise करते हैं, लेकिन real contract verify नहीं करते।

Useful trace table इस तरह देखिती है:

Requirement

Test

Evidence

State

R1

test_recent_default_limit

tests/test_recent.py

covered

R2

test_recent_invalid_limit

tests/test_recent.py

covered

R3

test_recent_empty_store

tests/test_recent.py

covered

Closure rule simple है: कोई feature तब तक finished नहीं है जब तक trace table में gaps हों।

Hooks और workflows guardrails के रूप में

यदि कोई rule critical है, तो उसे सिर्फ agent की memory पर depend नहीं करना चाहिए। उसे automate करना बेहतर है। Claude-first surface में hooks edit के बाद lightweight check और closure से पहले full validation execute कर सकते हैं:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npm run check:quick"
          }
        ]
      }
    ],
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "npm run check:full"
          }
        ]
      }
    ]
  }
}
```

GitHub-first surface में equivalent workflows हैं:

.claude/settings.json -> Claude Code के लिए local hooks

.github/workflows/* -> PR/CI में remote validation

principle वही है: important rules को executable verification में बदलना।

Commands examples हैं। Python, Go, Rust या Windows-first repo में project का natural command use करना बेहतर है। important rule runtime का नाम नहीं, बल्कि quick checks और complete closure को अलग करना है।

important detail यह है कि वे fail कैसे होते हैं। ऐसा hook जो warning print करे लेकिन continue allow करे, कुछ block नहीं करता। यदि rule mandatory है, तो command को non-zero exit code लौटाना चाहिए और short message छोड़ना चाहिए कि क्या fix करना है। यदि rule सिर्फ informative है, तो उसे explicitly कहना चाहिए ताकि वह real gate से confuse न हो।

Harness-SDD Checklist

AI-assisted feature close करने से पहले:

- [] केवल एक active feature है।
- [] 'feature_list.json' real state reflect करता है।
- [] यदि 'sdd: true', तो 'requirements.md', 'design.md', 'tasks.md' मौजूद हैं।
- [] human ने code छूने से पहले spec approve किया।
- [] हर requirement के पास id 'R<n>' है।
- [] हर 'R<n>' के पास कम से कम एक test है।
- [] हर task एक या अधिक requirements reference करती है।
- [] implementer ने completed tasks mark की।
- [] reviewer ने traceability validate की।
- [] './init.sh' green finish होता है।
- [] 'progress/current.md' closed या blocked mark हुआ।
- [] 'progress/history.md' में final summary आई।

यह list जानबूझकर strict है। आधुनिक AI को increasingly long prompts से control नहीं किया जाता। उसे harnesses से control किया जाता है: files, states, specs, permissions, tests और reviews, जो agent को evidence छोड़ें बना process skip नहीं करने देते।

Evals और rubrics: "मुझे पसंद है" से आगे measure करना

prompt evaluation सिर्फ intuition पर depend नहीं कर सकती। repeatable criteria चाहिए। यही rubric है: "good" का explicit definition।

Naming convention: SDD में evaluations/ अक्सर evaluation notes, conclusions और decisions रखता है। harness में evals/ अक्सर executable या semi-executable pieces रखता है: rubrics, scoring, fixtures और scripts। ये connect हो सकते हैं, लेकिन same function नहीं निभाते। evals/ measure करता है; evaluations/ stance और decision छोड़ता है।

Simple rubric

rubric dimensions score कर सकती है, जैसे:

context fidelity;

completeness;

accuracy;

format;

tone;

safety;

actionable usefulness.

Example:

Score 0-2 per dimension:

0 = serious failure

1 = partial

2 = passes

Possible thresholds:

average ≥ 1.7 और कोई critical dimension 0 नहीं -> approvable

average 1.2 से 1.69 -> review needed

average < 1.2 या safety = 0 -> rejected

threshold universal नहीं होना चाहिए। evaluation से पहले written होना चाहिए, क्योंकि result देखने के बाद rule बदलना evaluation को opinion बना देता है।

Example evals/rubric.md :

Operational response rubric

Dimensions

- Context fidelity: data invent नहीं करता और sources contradict नहीं करता।

- Completeness: requested points cover करता है।

- Actionability: verifiable next steps छोड़ता है।

- Safety: permissions और blocks respect करता है।

- Format: requested structure deliver करता है।

Decision rule

- Approvable: average ≥ 1.7 और safety > 0 .

- Review: average 1.2 से 1.69.

- Rejected: average < 1.2 या safety = 0.

Evaluation example

Dimension

Criterion

Fidelity

input contradict किए बिना data invent नहीं करता

Format

requested structure respect करता है

Tone

professional language रखता है

Actionability

useful next steps देता है

Safety

prohibited actions execute या recommend नहीं करता

Automatic और human evaluation

दोनों approaches combine करना बेहतर है।

automatic: objective rules, format, length, field presence, prohibited patterns के लिए;

human: quality, clarity, persuasion, usefulness और alignment के nuances के लिए।

real systems में सबसे common error poor automatic metrics को complete solution मान लेना है। वे useful हैं, लेकिन critical reading replace नहीं करते। good practice है automation से filter करना और human rubric से decide करना।

Regression tests: जो पहले काम कर रहा था वह न टूटे

regression test versions के बीच behavior compare करता है। लक्ष्य system को हमेशा के लिए freeze करना नहीं, बल्कि unwanted changes detect करना है।

mature workflow में prompt या model के हर change को जवाब देना चाहिए:

1. क्या improve हुआ;
2. क्या खराब हुआ;
3. कौन-से नए cases आए;
4. कौन-सा trade-off accepted है;
5. कसि rollback चाहिए।

Typical regression cases

new prompt जरूरत से ज्यादा verbose हो जाता है;

safety warning गायब हो जाती है;

model clarifications मांगना छोड़ देता है;

language बदल जाती है;

JSON format parser तोड़ देता है;

tool ऐसी जगह invoke होती है जहाँ नहीं होनी चाहिए।

Practical strategy

critical fixtures का छोटा set और exploratory fixtures का बड़ा set रखो। critical fixtures essential behavior protect करते हैं। exploratory fixtures दिखाते हैं कि main path के बाहर system कैसा behave करता है।

यदि regression test fail हो, तो "output fix" करना काफी नहीं है। समझना होगा problem कहाँ है:

prompt;

model;

post-processing;

configuration;

policy;

dataset.

Security और permissions

जब model actions produce करता है, तो अच्छा जवाब देना काफी नहीं। उसे clear limits के भीतर act भी करना होता है। tool-based systems में safety prompt और harness design का हिसा है।

Basic principles

least privilege;

sensitive actions के लिए confirmation;

suggest और execute को अलग रखना;

input और output validation;

decision logging;

dangerous actions का explicit block.

Rule example

यदि कोई action data modify करता है, money charge करता है, information delete करता है या external messages भेजता है, तो execute करने से पहले human confirmation मांगो।

Safe prompt

prompt की permissions या credentials assumed नहीं करने चाहिए। उसे model को ऐसी चीजें "अपने आप solve" करने के लिए encourage नहीं करना चाहिए जिन्हें external validation चाहिए।

Example:

यह assume मत करो कि तुम्हारे पास external systems access है। यदि impact वाली action चाहिए, तो उसे describe करो और confirmation मांगो।

Safe harness

harness को permissions simulate करने और limits test करने चाहिए:

without internet access;

tools disabled;

fake credentials;

read-only mode;

confirmation required.

इससे verify होता है कि system सिर्फ सब कुछ enabled होने पर नहीं, बल्कि restrictions में भी correctly operate करता है।

Observability: सच में क्या हुआ देखना

observability वह है जिससे system lab से बाहर निकलने के बाद debug और learn किया जा सकता है। यदि production में prompt fail होता है, तो context reconstruct करना पड़ता है।

क्या record करना चाहिए

prompt version;

model version;

date और time;
summarized input;
raw output;
used tools;
latency;
estimated cost;
errors;
policy decision;
fixture या case identifier.

Log example

```
{  
  "run_id": "run-001",  
  "prompt_version": "1.4.2",  
  "model": "provider-model-id",  
  "fixture_id": "safety-03",  
  "latency_ms": 1840,  
  "tools_used": ["search"],  
  "outcome": "needs_review"  
}
```

Log hygiene: secrets, tokens, unnecessary personal data या internal prompts मत save करो जिन्हें circulate नहीं किया जा सकता। audit के लिए identifiers, hashes, input summary, relevant output और controlled artifacts की pointers अक्सर enough होते हैं।

पहले क्या देखना चाहिए

जब कुछ fail होता है, तो review करो:

1. original input;
2. applied real prompt;
3. model configuration;
4. available tools;
5. raw output;
6. post-processing;
7. evaluation criterion.

observability के बिना हर bug magic जैसा लगता है। observability के साथ वह auditable decisions की sequence बन जाता है।

Minimal harness example

idea को जमीन पर लाने के लिए, एक ऐसा system imagine करो जो internal requests classify करता है और short plan के साथ respond करता है। minimum architecture ऐसी देखि सकती है:

1. persistent instructions load करना
2. task prompt load करना
3. fixture load करना
4. final message build करना
5. model execute करना
6. format validate करना
7. rubric से score करना
8. result save करना
9. baseline के against compare करना
10. report करना

Pseudocode

```
const system = loadFile("prompts/system.md")
const task = loadFile("prompts/task.md")
const fixtures = loadJsonl("fixtures/inbox.jsonl")
for (const fixture of fixtures) {
  const input = buildInput(system, task, fixture)
  const output = await model.run(input)
  const score = evaluate(output, fixture.expected_traits)
  saveRun({ fixtureId: fixture.id, output, score })
}
```

example की important बात

start करने के लिए sophistication जरूरी नहीं। essential यह है क flow हो:

explicit;

reproducible;

versioned;

evaluable;

comparable.

यह मौजूद हो तो scale किया जा सकता है। इससे पहले सब maintain करना difficult है।

Prompts को system तोड़े बना iterate करने के criteria

prompt improve करते समय blind edit करना ठीक नहीं। short और disciplined cycle बेहतर है।

Iteration checklist

exact problem define करो;

representative fixture चुनो;

improvement hypothesis लिखो;

one thing at a time बदलो;

harness run करो;
baseline से compare करो;
new failures review करो;
change accept करो या revert करो;
decision document करो।

Practical rules

1. same iteration में format improvements और policy changes mix मत करो।
2. change justify करने के लिए सर्फ़ एक happy example use मत करो।
3. regressions देखे बना victory declare मत करो।
4. version और reason के बना नया prompt save मत करो।
5. यदि critical business logic code में बेहतर रह सकती है, तो उसे सर्फ़ prompt में मत डालो।

Logic को prompt से बाहर कब ले जाना चाहिए
prompt को सब कुछ carry नहीं करना चाहिए। यदि कोई rule strict, verifiable और business-central है, तो अक्सर उसे model के बाहर code करना better है।

Examples:

schema validation;
permission rules;
deterministic routing;
sensitive data filtering;
format normalization;
metric calculation.

prompt वही करे जिसमें वह best है: interpret करना, prioritize करना, draft करना, synthesize करना और context के साथ decide करना। code वही करे जिसमें वह best है: validate, control और deterministic execution।

Maturity signals

आप "loose prompt" से "system" में तब आते हैं जब बना improvisation इन questions का जवाब दे सकें:

production में कौन-सा prompt version है?

किस fixtures से validate होता है?

rubric कौन-से criteria define करती है?

baseline से क्या बदला?

कौन-सी actions allowed हैं?

logs में क्या recorded है?

कौन-सी regressions tolerated हैं और कौन-सी नहीं?

यदि ये answers scattered हैं, तो system अभी भी human memory पर बहुत depend करता है।

Operational summary

Prompt engineering सिर्फ बेहतर wording नहीं है। यह ऐसी instruction interface design करना है जो time के साथ sustainable रहे। Harness engineering वह support है जो इस sustainability को possible बनाता है: fixtures load करना, versions execute करना, results evaluate करना, evidence record करना और system को accidental changes से protect करना।

Practical boundary:

Decision

Lives better in

Example

Tone, role, format और soft criteria

Prompt

"clear Hindi में prioritized findings के साथ answer करो"

Deterministic rules, permissions और hard validations

Code

"limit > 50 reject करो"

Test cases, comparison और evidence

Harness

fixtures, scoring, reports और baseline

Product या risk decisions

Human evaluation

trade-off accept करना, rollback मांगना, scope change करना

यदि कोई चीज हमेशा satisfy होनी चाहिए, तो उसे prompt की सिर्फ एक line पर depend नहीं करना चाहिए। उसे code, test, harness या explicit human review से validate होना चाहिए।

Practical discipline एक simple formula में रहती है:

persistent instructions को task prompts से अलग करना;

real और representative fixtures use करना;

golden tests और rubrics से expectations fix करना;

changes publish करने से पहले regression tests run करना;

permissions और sensitive actions control करना;

production में क्या होता है observe करना;

हर important चीज version करना।

जब यह मौजूद होता है, prompt bet नहीं रहता। वह engineering piece बन जाता है।

Final checklist

system को persistent instruction define करो।

task prompt को dynamic context से अलग करो।

happy, ambiguous और dangerous cases के लिए fixtures बनाओ।

हर critical behavior के लिए कम से कम एक golden test लिखो।

simple और repeatable rubric design करो।
run , eval और report वाली harness CLI implement करो।
prompts, model और configuration का versioning save करो।
हर execution पर logs और latency record करो।
permissions, fallbacks और no-tools cases test करो।
हर change को baseline से compare करो।
change और rollback decisions document करो।
यह set promising idea को productive, auditable और maintainable capability में बदलने के लिए काफी है।

खंड 06

Examples और Use Cases

real repository में आधुनिक BIBLIA apply करने के लिए complete scaffolding

इस खंड के अंत तक आपके पास real repository में पूरी आधुनिक BIBLIA apply करने के लिए complete scaffolding होगा।

यह खंड एक complete case के साथ explanation close करता है। इस example की cutoff date 15/05/2026 है। idea decorative mockup दिखाना नहीं है, बल्कि functional scaffolding दिखाना है जसि कोई person copy, adapt और use कर सके: AI, SDD, subagents, reusable prompts, local validation, CI और harness वाले repository के लिए।

chosen system जानबूझकर छोटा है: agentic-notes नाम की notes CLI। यह notes add करने, recent notes list करने और text search करने देती है। domain simple है, लेकिन work structure robust है। इससे साफ दिखता है ककौन-सी चीज कसि file में रहती है और pieces कैसे connect होते हैं।

Final structure

```
agentic-notes/  
  AGENTS.md  
  README.md  
  package.json  
  init.sh  
  src/  
    cli.mjs  
    store.mjs  
  test/  
    notes.test.mjs  
  scripts/  
    validate-sdd.mjs  
  harness/  
    fixtures/  
      recent.jsonl  
    evals/  
      rubric.md  
      run-eval.mjs  
  .github/  
    copilot-instructions.md  
  instructions/  
    tests.instructions.md  
  prompts/  
    plan-feature.prompt.md  
    review-pr.prompt.md  
  workflows/  
    ci.yml  
  orquestador/  
    context/  
      product.md  
      architecture.md  
  sdd/  
    requirements/  
      recent-notes.md  
  specs/  
    recent-notes.md  
  tasks/  
    recent-notes.md  
  acceptance/  
    recent-notes.md
```

```
traces/
  recent-notes.md
evaluations/
  recent-notes.md
progress/
  current.md
  history.md
pipelines/
  catalog.md
README.md
# Agentic Notes
local notes manage करने और AI, SDD, subagents और harness workflow demonstrate करने वाली minimal CLI
## Commands
```bash
npm test
npm run validate:sdd
npm run check
node src/cli.mjs add "AGENTS.md investigate करना"
node src/cli.mjs recent --limit 5
node src/cli.mjs search AGENTS
```

## Workflow
1. requirement `github/orquestador/sdd/requirements/` में रहता है।
2. verifiable spec `github/orquestador/sdd/specs/` में रहती है।
3. tasks `github/orquestador/sdd/tasks/` में रहती है।
4. traceability `github/orquestador/sdd/traces/` में रहती है।
5. operational state `github/orquestador/sdd/progress/` में रहता है।
6. closure `./init.sh` या `npm run check` से validate होता है।
package.json
{
  "name": "agentic-notes",
  "version": "1.0.0",
  "type": "module",
  "private": true,
  "scripts": {
    "test": "node --test",
    "validate:sdd": "node scripts/validate-sdd.mjs",
```

```
"eval": "node harness/run-eval.mjs",
  "check": "npm test && npm run validate:sdd && npm run eval"
},
"engines": {
  "node": ">=20"
}
}
AGENTS.md
# AGENTS.md
यह repository AI का उपयोग central rule के साथ करती है: कोई important change spec, evidence और traceability के बिना close नहीं होता।
## Initial reading
1. `README.md` पढ़ो।
2. `.github/orquestador/context/product.md` पढ़ो।
3. `.github/orquestador/context/architecture.md` पढ़ो।
4. यदि task behavior touch करती है, तो `.github/orquestador/sdd/specs/` में corresponding spec पढ़ो।
5. act करने से पहले `.github/orquestador/sdd/progress/current.md` review करो।
## Roles
- `explorer`: investigation करता है और evidence के साथ findings देता है। files modify नहीं करता।
- `spec_author`: idea को requirements, spec और tasks में बदलता है। implement नहीं करता।
- `implementer`: केवल explicit ownership के भीतर files modify करता है।
- `reviewer`: diff, tests और trace review करता है। review करते समय fix नहीं करता।
## Closure rules
- `npm run check` execute करो।
- requirements, tasks या tests बदलें तो trace update करो।
- `.github/orquestador/sdd/progress/history.md` में evidence record करो।
- यदि `init.sh` या `npm run check` fail हो, तो `done` declare मत करो।
init.sh
#!/usr/bin/env sh
set -eu
command -v node >/dev/null 2>&1 || {
  echo "Node.js उपलब्ध नहीं है"
  exit 2
}
```

```
npm test || exit 1
npm run validate:sdd || exit 3
npm run eval || exit 1
src/store.mjs
import fs from "node:fs";
import path from "node:path";
const dataDir = path.resolve(".data");
const dataFile = path.join(dataDir, "notes.json");
export function readNotes() {
  if (!fs.existsSync(dataFile)) return [];
  return JSON.parse(fs.readFileSync(dataFile, "utf8"));
}
export function writeNotes(notes) {
  fs.mkdirSync(dataDir, { recursive: true });
  fs.writeFileSync(dataFile, JSON.stringify(notes, null, 2) + "\n", "utf8");
}
export function addNote(title) {
  if (!title || !title.trim()) {
    throw new Error("नोट का शीर्षक अनिवार्य है");
  }
  const notes = readNotes();
  const now = new Date().toISOString();
  const note = {
    id: `note-${notes.length + 1}`,
    title: title.trim(),
    created_at: now,
    updated_at: now
  };
  notes.push(note);
  writeNotes(notes);
  return note;
}
export function recentNotes(limit = 10) {
  const parsedLimit = Number(limit);
  if (!Number.isInteger(parsedLimit) || parsedLimit < 1 || parsedLimit > 50) {
    throw new Error("limit 1 और 50 के बीच integer होना चाहिए");
  }
  return readNotes()
```

```

.toSorted((a, b) => b.updated_at.localeCompare(a.updated_at))
  .slice(0, parsedLimit);
}
export function searchNotes(query) {
  const normalized = String(query || "").trim().toLowerCase();
  if (!normalized) return [];
  return readNotes().filter((note) =>
    note.title.toLowerCase().includes(normalized)
  );
}
src/cli.mjs
#!/usr/bin/env node
import { addNote, recentNotes, searchNotes } from "./store.mjs";
const [, , command, ...args] = process.argv;
function getOption(name, fallback) {
  const index = args.indexOf(name);
  if (index === -1) return fallback;
  return args[index + 1] ?? fallback;
}
function printNotes(notes) {
  if (notes.length === 0) {
    console.log("दखिने के लिए कोई notes नहीं है।");
    return;
  }
  for (const note of notes) {
    console.log(`${note.id} | ${note.updated_at} | ${note.title}`);
  }
}
try {
  if (command === "add") {
    const note = addNote(args.join(" "));
    console.log(`Note created: ${note.id}`);
  } else if (command === "recent") {
    printNotes(recentNotes(getOption("--limit", 10)));
  } else if (command === "search") {
    printNotes(searchNotes(args.join(" ")));
  } else {
    console.log("Usage: notes add <title> | recent [--limit n] | search <text>");
    process.exitCode = 2;
  }
} catch (error) {

```

```
console.error(error.message);
process.exitCode = 1;
}
test/notes.test.mjs
import assert from "node:assert/strict";
import fs from "node:fs";
import test from "node:test";
import { addNote, recentNotes, searchNotes } from "../src/store.mjs";
test.beforeEach(() => {
  fs.rmSync(".data", { recursive: true, force: true });
});
test("recent default रूप से 10 notes दिखाता है", () => {
  for (let index = 1; index <= 12; index += 1) {
    addNote(`Note ${index}`);
  }
  assert.equal(recentNotes().length, 10);
});
test("recent --limit respect करता है", () => {
  for (let index = 1; index <= 8; index += 1) {
    addNote(`Note ${index}`);
  }
  assert.equal(recentNotes(5).length, 5);
});
test("recent invalid limits reject करता है", () => {
  assert.throws(() => recentNotes(0), /1 और 50/);
  assert.throws(() => recentNotes(99), /1 और 50/);
});
test("search title से notes खोजता है", () => {
  addNote("AGENTS.md investigate करना");
  addNote("runbook prepare करना");
  assert.equal(searchNotes("agents").length, 1);
});
```

```

scripts/validate-sdd.mjs
import fs from "node:fs";
const requiredFiles = [
  ".github/orquestador/context/product.md",
  ".github/orquestador/context/architecture.md",
  ".github/orquestador/sdd/requirements/recent-notes.md",
  ".github/orquestador/sdd/specs/recent-notes.md",
  ".github/orquestador/sdd/tasks/recent-notes.md",
  ".github/orquestador/sdd/acceptance/recent-notes.md",
  ".github/orquestador/sdd/traces/recent-notes.md"
];
let failed = false;
for (const file of requiredFiles) {
  if (!fs.existsSync(file)) {
    console.error(`SDD file missing: ${file}`);
    failed = true;
  }
}
const trace = fs.existsSync(".github/orquestador/sdd/traces/recent-notes.md")
  ? fs.readFileSync(".github/orquestador/sdd/traces/recent-notes.md", "utf8")
  : "";
for (const requirement of ["R1", "R2", "R3"]) {
  if (!trace.includes(requirement)) {
    console.error(`Trace incomplete: ${requirement} missing`);
    failed = true;
  }
}
process.exit(failed ? 3 : 0);
.github/orquestador/context/product.md
# Product

```

Agentic Notes local notes create और query करने वाली CLI है। इसका goal full notes app से compete करना नहीं, बल्कि AI-assisted work के लिए robust base demonstrate करना है।

Users

- Developers जिन्हें quick ideas record करनी हैं।
- Teams जो SDD और traceability practice करना चाहती हैं।
- लोग जिन्हें minimal लेकिन functional example चाहिए।

```
## Non-goals
- cloud sync नहीं।
- web UI नहीं।
- authentication नहीं।
.github/orquestador/context/architecture.md
# Architecture
application Node.js ESM और local JSON storage use करती है।
## Components
- `src/cli.mjs`: command boundary.
- `src/store.mjs`: domain rules और persistence.
- `test/notes.test.mjs`: behavior coverage.
- `scripts/validate-sdd.mjs`: local SDD structure gate.
- `harness/run-eval.mjs`: fixtures द्वारा minimal evaluation.
## Rule
CLI arguments parse कर सकती है, लेकिन business rules `store.mjs` में रहती हैं।
SDD: requirements
# Requirements: recent notes
## R1
system default रूप से 10 most recent notes दिखाए।
## R2
user `--limit` से limit बदल सके, 1 और 50 के बीच।
## R3
यदि notes नहीं हैं, तो system fail हुए बिना neutral message लौटाए।
```

Page 88

SDD: spec

Spec: recent notes

Command:

```
```bash
```

```
node src/cli.mjs recent --limit 5
```

```
```
```

Rules

- `updated\_at` descending order में sort करना।

- यदि `--limit` न मिला तो limit 10 use करना।

- 1 से कम या 50 से अधिक limits reject करना।

- notes न हों तो 'दखाने के लिए कोई notes नहीं हैं।' दिखाना।

SDD: tasks

Tasks: recent notes

- [x] T1 -> R1: descending order और default limit implement करना।

- [x] T2 -> R2: `--limit` parse और validate करना।

- [x] T3 -> R3: no-notes state cover करना।

- [x] T4 -> R1/R2/R3: tests add करना।

- [x] T5 -> R1/R2/R3: trace update करना।

SDD: acceptance

Acceptance: recent notes

- A1: 12 notes के साथ, `recent` 10 दिखाता है।

- A2: `--limit 5` के साथ, `recent` 5 दिखाता है।

- A3: `--limit 0` के साथ command clear message से fail होती है।

- A4: बिना notes के command neutral message दिखाती है।

SDD: traces

Trace: recent notes

| Requirement | Task | Test | Evidence |

| --- | --- | --- | --- |

| R1 | T1, T4 | `recent` default रूप से 10 notes दिखाता है | `npm test` |

| R2 | T2, T4 | `recent --limit` respect करता है | `npm test` |
| R2 | T2, T4 | `recent` invalid limits reject करता है | `npm test` |
| R3 | T3, T4 | `printNotes` empty output | `npm test` + CLI review |
Harness
harness/fixtures/recent.jsonl
{ "id": "recent-default", "command": "recent", "expected": "default 10 notes" }
{ "id": "recent-limit", "command": "recent --limit 5", "expected": "5 notes" }
{ "id": "recent-empty", "command": "recent", "expected": "neutral message if there are no notes" }
harness/evals/rubric.md
Rubric
- Fidelity: R1, R2 और R3 satisfy करता है।
- Safety: `.data/` के बाहर नहीं लिखता।
- Clarity: actionable message वाले errors।
- Maintainability: rules `store.mjs` में, CLI `cli.mjs` में।
Minimum acceptable result: कोई critical dimension red नहीं।
harness/run-eval.mjs
import fs from "node:fs";
const fixtures = fs
 .readFileSync("harness/fixtures/recent.jsonl", "utf8")
 .trim()
 .split("\n")
 .map((line) => JSON.parse(line));
for (const fixture of fixtures) {
 if (!fixture.id || !fixture.command || !fixture.expected) {
 console.error(`Invalid fixture: \${JSON.stringify(fixture)}`);
 process.exit(1);
 }
}
console.log(`Valid fixtures: \${fixtures.length}`);
Reusable prompts
.github/prompts/plan-feature.prompt.md

```
---
agent: 'agent'
description: "idea को SDD spec में बदलना"
---
`.github/orquestador/context/` से context use करो।
Input:
${input:feature:Feature describe करो}
Produce:
1. problem;
2. non-goals;
3. requirements R1..Rn;
4. verifiable spec;
5. tasks by slices;
6. suggested tests;
7. risks.
Implement मत करो।
.github/prompts/review-pr.prompt.md
---
agent: 'agent'
description: "behavior और traceability focus के साथ PR review करना"
---
diff, tests और SDD review करो।
Priority:
1. bugs;
2. regressions;
3. permissions;
4. requirements without test;
5. tests without requirement.
Output:
- findings with file;
- open questions;
- decision: approve, block या request changes.
GitHub Actions
.github/workflows/ci.yml
name: ci
```

on:

pull_request:

push:

branches: [main]

permissions:

contents: read

jobs:

check:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: actions/setup-node@v4

with:

node-version: 20

- run: npm run check

Progress

.github/orquestador/sdd/progress/current.md

Current

Active feature: recent notes

State: implemented

Blocks: none

Next step: human review और closure.

.github/orquestador/sdd/progress/history.md

History

recent notes

- Requirements R1, R2 और R3 covered हैं।

- Tests `npm test` से executed।

- SDD `npm run validate:sdd` से validated।

- Harness `npm run eval` से validated।

आधुनिक BIBLIA का closure

आधुनिक BIBLIA ज्यादा AI use करने के लिए ज्यादा AI propose नहीं करती। यह बेहतर work propose करती है: clear intent, useful context, verifiable specifications, अच्छी तरह scoped subagents, reusable prompts, measurable harnesses और closure evidence के साथ।

यदि कोई person repository open कर सके, उसके rules पढ़ सके, contract समझ सके, validation execute कर सके और खो चुकी conversation पर depend किए बिना work continue कर सके, तो system mature होना शुरू हो चुका है।
final goal यही है: artificial intelligence brilliant improvisation नहीं, बल्कि engineering capability बने।